
StratEx: Typed Strategy Programs for Bayesian Exploration in LLM-Based Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 As LLM agents move from single-turn prompting to long-horizon coding, web-
2 navigation, and tool-use workflows, their success increasingly depends on reusable
3 strategies rather than isolated actions. However, existing agent scaffolds explore
4 over low-level actions, prompt modes, or untyped skill names, which operates at
5 the wrong abstraction level for compositional procedures. We manifest this limita-
6 tion as *strategy program collapse*: the posterior selection distribution concentrates
7 on a few initial programs while higher-value composite procedures remain unex-
8 plored, and the combinatorial space of possible programs makes discovery without
9 structural assumptions intractable. We introduce **StratEx**, a strategy-exploration
10 scaffold that lifts the bandit arm from an action to a *typed strategy program*: an
11 auditable DSL AST with primitive schemas, verifier-mediated control flow, and
12 a finite library expanded from coherent training traces. StratEx selects complete
13 programs with Beta-Bernoulli Thompson sampling while an auxiliary controller
14 handles within-program action stochasticity. Our analysis separates finite-library
15 selection from library induction, showing the exponential enumeration barrier
16 faced by action-level random exploration and the conditions under which trace
17 mining can admit useful composite programs. Across SWE-bench Lite, WebArena,
18 and ToolBench-IR evaluations, StratEx improves task success over action-level
19 Thompson Sampling and standard exploration baselines. StratEx establishes typed
20 strategy programs as a viable exploration abstraction for LLM-agent scaffolds and
21 motivates further research into program-level exploration mechanisms.

22 1 Introduction

23 Modern LLM-based agents can perform complex, multi-step tasks in domains like autonomous soft-
24 ware engineering [30, 12], web navigation [38, 7], and tool-augmented reasoning [19, 25, 21, 14, 15,
25 31]. Scientific and research agents push this trend further: recent systems evaluate seminar-grounded
26 research workflows, evidence-based chemistry reasoning, scientific-context protocols, and reusable
27 multi-agent optimization skills [29, 33, 11, 32]. These systems typically add a *scaffold* around the
28 frozen LLM to improve task performance: the scaffold may include strategy libraries, memory mod-
29 ules, self-reflection mechanisms, and learned controllers. Unlike traditional reinforcement learning
30 (RL) with fixed reward signals, scaffold-level adaptation allows the agent to evolve its reasoning
31 procedures without fine-tuning the underlying model weights.

32 A core unsolved problem in scaffold design is **strategy exploration**: how should an agent discover
33 and reuse useful problem-solving procedures when these procedures are compositional programs
34 rather than atomic actions? In traditional RL, exploration operates over discrete or continuous action
35 spaces [26, 28, 1]. In many agent scaffolds, exploration targets a fixed pool of named strategies
36 or prompt modes. Both views choose the wrong abstraction level for many agentic workloads. A

successful SWE-bench routine may be “decompose the issue, retrieve relevant files, simulate a failure path, patch minimally, then verify the fix.” A successful WebArena routine may branch on website state and backtrack after a failed form submission. These are not merely longer action strings. They are reusable *procedures* with typed inputs, intermediate evidence, verifier checks, and recovery structure. We therefore treat the exploration unit itself as an executable program.

The challenge is that standard exploration strategies are designed for stationary action spaces. Fixed schedules and curiosity rewards encourage new tool calls or states, but they do not ask whether the agent has composed a new reasoning procedure. Early successes with simple strategies create anchored value estimates that suppress exploration of more sophisticated approaches. Scaffold-level adaptation is then limited to refining within a small strategy neighborhood, unable to discover higher-value compositional procedures. We formally define this failure mode as *strategy program collapse*: the posterior or empirical selection distribution concentrates on an early program while a higher-value composite program remains insufficiently sampled.

To address these challenges, we propose **StratEx**, a strategy exploration framework that lifts the exploration unit from individual actions to executable typed strategy programs. StratEx maintains a Beta-Bernoulli posterior over each program in a finite library and applies Thompson sampling to select complete procedures, while an adaptive controller handles within-program action stochasticity. The theory separates program selection (exponential advantage over action-level once in library) from program induction (the bottleneck requiring heuristics).

We summarize our contributions:

- **Typed Strategy Programs as the Exploration Unit.** We introduce typed executable programs as the selection target for LLM-agent scaffolds, turning compositional reasoning procedures into selectable, auditable, reusable objects.
- **Theory Separating Selection from Induction.** We prove that program-level Thompson sampling removes the exponential enumeration barrier of action-level exploration, while characterizing the library-induction bottleneck as the remaining open problem.
- **Controlled Evaluation Under Matched Conditions.** Under a same-backbone protocol, StratEx achieves consistent improvements over action-level baselines across SWE-bench Lite, WebArena, and ToolBench-IR. Ablation and timing controls attribute the gains to offline program-level exploration.

2 Related Work

We situate StratEx within exploration theory, temporal abstraction, and LLM-agent scaffolds. The closest abstract view of StratEx is a *bandit over executable options*: each arm is a typed DSL AST that conditions the frozen LLM agent’s low-level behavior. The contribution is to make this abstraction operational for modern LLM agents: programs have typed schemas, verifier-mediated control flow, benchmark adapters, induction from traces, and posterior-guided reuse.

Exploration in Bandits and Reinforcement Learning. Classical exploration methods such as ϵ -greedy, UCB, and Thompson sampling operate over actions or fixed arms [26, 28, 1, 20]. In deep RL, intrinsic motivation methods define rewards based on novelty: ICM [16] uses forward dynamics prediction error, RND [3] measures error against a random target network, and VIME [8] uses information gain about dynamics. Recent work extends this diagnosis to language agents: *LLMs are Greedy Agents* [22] studies the knowing-doing gap with ϵ -greedy and self-correction on bandit-style tasks; ProCeedRL [6] introduces process-level critics and exploratory demonstrations for agentic reasoning; and Rewarding the Rare [9] explicitly rewards correct but rare high-level strategies at the rollout level. StratEx uses the same Bayesian exploration principle as Thompson sampling, but moves the arm definition from raw action choices to typed strategy programs. This distinction matters because a program arm can encode a multi-step routine such as retrieve–simulate–verify, whereas action-level exploration must rediscover the ordered sequence through low-level samples.

Temporal Abstraction and Structured Options. Options and hierarchical RL learn temporally extended behaviors with initiation sets, policies, and termination rules [27, 2]. StratEx is conceptually close to this family: a strategy program can be read as an option whose internal policy is implemented by a frozen LLM plus a typed interpreter. Program synthesis and prompt optimization similarly search over structured artifacts that control behavior, while contextual bandits and Bayesian optimization

over prompts, tools, or policies choose among discrete strategy choices. The technical boundary from these lines of work is the representation and learning layer. Options usually learn low-level policies in an environment; StratEx operates at the scaffold level, where the option body is an auditable AST over reasoning primitives, branch conditions, verifier calls, and benchmark adapters. Each candidate is a typed AST interpreted into primitive-level prompts and benchmark-valid actions, not merely a prompt string or a policy label. This representation gives the selector a semantically meaningful unit—a procedure with evidence, verification, and recovery slots—and gives the artifact an auditable object beyond a natural-language prompt. Program-level Thompson sampling is therefore best viewed as bandit selection over structured options, with library expansion providing arm construction. Our theory formalizes this representation advantage under finite-library and induction assumptions.

Self-Improving LLM Agents and Skill Libraries. ReAct [35], Reflexion [25], Voyager [30], ChatDev [18], and SWE-agent [34] show that LLM agents can improve through prompts, memory, reflection, tools, and reusable skills without updating LLM weights. Recent systems make this scaffold boundary much more concrete. SE-Agent revisits and recombines pilot trajectories to self-evolve on SWE-bench Verified [13]; R2E-Gym scales open-weight SWE agents with procedurally curated environments and hybrid verifiers [10]; SkillWeaver turns website interaction traces into reusable APIs [36]; WALT reverse-engineers latent website functionality into invocable tools [17]; WebEvolver adds a co-evolving world model for sustained web self-improvement [4]; and DeepResearcher trains deep research agents end-to-end in real web environments [37]. These systems are strong adjacent SOTA baselines for coding and web tasks, but they still optimize concrete trajectories, tools, or skills rather than posterior-guided selection over typed strategy programs. Recent work adds several adjacent pressures: test-time tool evolution and financial tool-use evaluation expose the need for robust tool adaptation [14, 15, 31]; scientific-agent infrastructure and benchmarks stress long-horizon research workflows [29, 11]; and multi-agent domain systems distill repeated experience into reusable skills [33, 32]. Voyager-style libraries store skills or code fragments and select them heuristically. StratEx instead stores strategy procedures as typed DSL programs with explicit input/output signatures and posterior uncertainty over episode-level success. This makes the selection unit executable and comparable across tasks, while still relying on heuristic induction to propose new candidate programs.

3 StratEx: Bayesian Strategy Exploration

StratEx addresses strategy program stagnation through three linked mechanisms: (1) represent reusable reasoning routines as *typed strategy programs*; (2) maintain a Beta-Bernoulli posterior over the success probability of each finite-library program and select complete programs with Thompson sampling; and (3) adapt low-level action stochasticity via a learned controller. Figure 1 shows the overall architecture. The method separates *program induction* (constructing candidate ASTs from training traces) from *program selection* (choosing among the current finite library).

3.1 Setup, State, and Strategy DSL

We consider a task distribution $\mathcal{D}$ and a typed strategy DSL with six atomic primitives {Decompose, Retrieve, Verify, Simulate, Reflect, Compress} plus two control-flow operators {Branch, Backtrack}. A *strategy program* z is a typed abstract syntax tree (AST) over these operations. It maps an agent state to an action guide and then to low-level tool actions. The agent maintains a posterior $p_t(z \mid \mathcal{H}_t)$ over programs given history $\mathcal{H}_t$, and executes a policy $\pi_\theta(a \mid s, z)$ conditioned on the sampled program.

Agent state and normalized action interface. At step t , the state is

$$s_t = (x, o_{\leq t}, m_t, b_t, \ell_t),$$

where x is the task specification, $o_{\leq t}$ is the observation/tool transcript, m_t is retrieved memory or evidence, b_t is the current belief summary, and ℓ_t records remaining budget and verifier status. For learning components, s_t is embedded by a frozen text encoder plus scalar budget/verifier features. The action space is benchmark-specific but exposed through a normalized discrete interface: search, read, edit, run_test, and submit for SWE-bench; click, type, scroll, navigate, and submit for WebArena; and call_tool, inspect, and answer for ToolBench-IR. Random action exploration samples only from actions valid under the current benchmark mask. When an action requires free-form arguments, such as a SWE-bench patch or a web form value, the frozen

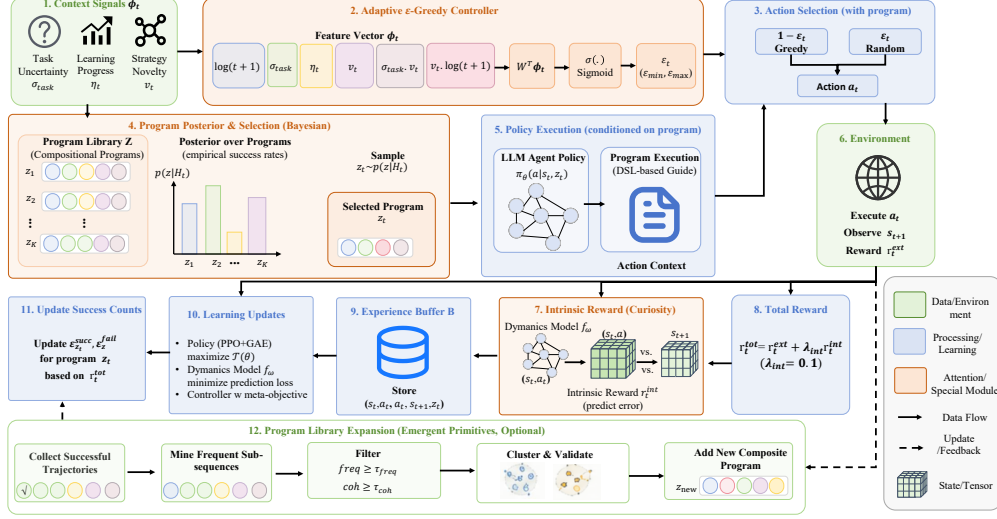


Figure 1: **StratEx Framework Architecture.** StratEx explores over composable strategy programs instead of actions alone, maintaining a Beta-Bernoulli posterior over each program’s success probability and selecting programs with Thompson sampling. An adaptive ϵ -greedy controller modulates action-level exploration based on context signals. The curiosity module provides auxiliary intrinsic rewards. Program library expansion identifies useful primitive sub-sequences from successful trajectories.

141 LLM generates those arguments from the AST-node output and current transcript; the learned policy
 142 selects the normalized action type and reranks candidate arguments rather than optimizing token-level
 143 LLM probabilities.

144 **Trainable policy modules.** We write $\pi_\theta(a | s, z)$ for the scaffold policy over the normalized discrete
 145 interface, not for the LLM decoding distribution. The parameter vector θ contains the action head,
 146 argument reranker, program encoder, and value head, but it excludes the frozen LLM. PPO log-
 147 probabilities are computed for scaffold decisions such as the selected normalized action, branch
 148 choice, and candidate-argument reranking score. The LLM is called as a conditional generator inside
 149 the interpreter, and gradients are stopped through all generated text and environment observations.
 150 All baselines use the same frozen backbone and normalized action interface.

151 **Strategy DSL.** A strategy program is a typed AST whose statements are atomic reason-
 152 ing primitives or verifier-mediated control flow. Atomic nodes are Decompose, Retrieve,
 153 Verify, Simulate, Reflect, and Compress; control-flow nodes are Branch(cond, z_{then} , z_{else}) and
 154 Backtrack(cond, z_{recover}). Each node exposes a typed JSON schema, so a program is not an unstruc-
 155 tured prompt label. It has explicit inputs, outputs, branch conditions, recovery behavior, and budget
 156 constraints. The type checker enforces AST depth $\leq D_{\text{max}}$, no recursive cycles, type-compatible
 157 edges, and benchmark-valid action masks. Full BNF grammar and primitive schemas are in the
 158 supplement.

159 **Prompt schema and AST interpreter.** Each AST node is interpreted by a de-
 160 terministic wrapper: {primitive, inputs, state_summary, allowed_actions, budget,
 161 output_schema}. The LLM returns JSON matching the primitive’s output type. DECOMPOSE
 162 returns subtasks, RETRIEVE returns evidence queries or tool calls, SIMULATE returns a candidate
 163 consequence score, VERIFY returns a boolean plus verifier rationale, REFLECT returns recovery notes,
 164 and COMPRESS returns a bounded state summary. BRANCH and BACKTRACK consume boolean out-
 165 puts from VERIFY, SIMULATE, or REFLECT; verifier-provided booleans override ungrounded LLM
 166 booleans whenever an executable verifier is available. Invalid JSON, type mismatches, and budget
 167 violations are rejected, logged as verifier failures for the current node, and routed through a bounded
 168 recovery path when the AST contains BACKTRACK. Program validity is checked by enforcing AST
 169 depth $\leq D_{\text{max}}$, no recursive cycles, type-compatible edges, and a maximum prompt-token budget.
 170 Additional prompt schemas, adapters, and trace fields are in the Supplementary Materials.

Program library expansion. StratEx expands the program library during offline training by mining contiguous operation sub-sequences from successful trajectories. For a candidate sub-sequence $u = (o_i, \dots, o_j)$, $\text{freq}(u)$ is the number of successful trajectories in the current training window that contain u , and $\text{coh}(u)$ is the mean pairwise cosine similarity of the program-encoder embeddings for all occurrences of u . A candidate enters the library only when $\text{freq}(u)$ and $\text{coh}(u)$ exceed learned thresholds and its embedding is sufficiently different from existing programs under cosine distance. New programs are initialized with the same symmetric Beta prior as other programs. Failed trajectories are not mined as positive program seeds; they contribute negative evidence only by incrementing the failure count of the sampled program. A fixed per-epoch insertion budget plus the novelty threshold prevents unbounded library growth. This expansion rule is heuristic; the supplement gives an idealized induction model under frequency-separation assumptions.

Scaffold evolution. The LLM backbone is frozen. During offline training, StratEx learns the scaffold around it: the program encoder, adaptive controller, value head, dynamics model, and an initial program library $\mathcal{Z}_0$ constructed only from training tasks. We therefore use “self-evolving” to mean scaffold-level adaptation rather than LLM weight updates. The trainable parameters are confined to these scaffold components, and the backbone itself is not fine-tuned.

Evaluation boundary. All benchmark results in Table 1 use a held-out pass@1 protocol. Before evaluation, the learned scaffold parameters and the initial library $\mathcal{Z}_0$ are frozen. For each benchmark task and seed, posterior counts and trajectory memory are reset, and the mutable library state is reset to $\mathcal{Z}_0$. The task receives one episode-level attempt. The terminal success/failure update is logged after the final answer and therefore cannot improve that task’s pass@1 outcome; it is discarded before the next held-out task. Thus evaluation-time Thompson sampling acts as an uncertainty-aware randomized selector over the frozen library at episode start, while the main benefit of program-level posterior feedback is realized during offline scaffold training, where many training episodes shape the library, program-conditioned policy/value heads, and controller. No trajectory, count, memory item, or discovered program is transferred across held-out tasks or seeds. The agent cannot access reference fixes, hidden tests, public patches, benchmark hints, or previous evaluation trajectories. Program-discovery and training-analysis tables in the supplement summarize offline training runs and are not transductive updates across held-out tasks.

3.2 Program Posterior and Thompson Sampling

We maintain a Beta posterior over the success probability of each program in the finite library $\mathcal{Z}$. This is an online belief-updating process over a discrete program space. Sampling proceeds by drawing one success-probability sample for every program and selecting the program with the largest draw. The induced selection distribution over programs is therefore the Thompson-sampling distribution, not a Laplace-smoothed distribution over success counts.

Definition 3.1 (Program Posterior). The prior is uniform: $p(z) = 1/|\mathcal{Z}|$ for all $z \in \mathcal{Z}$. After observing history $\mathcal{H}_t = \{(z_i, r_i)\}_{i=1}^{t-1}$, we maintain a Beta posterior over each program’s success probability θ_z . Sampling proceeds by drawing $\theta_z \sim \text{Beta}(\alpha + c_z^{\text{succ}}, \beta + c_z^{\text{fail}})$ for each program z , then selecting $\arg \max_z \theta_z$:

$$\theta_z \sim \text{Beta}(\alpha + c_z^{\text{succ}}, \beta + c_z^{\text{fail}}), \quad c_z^{\text{succ}} = \sum_{i: z_i = z} \mathbb{I}[r_i = 1], \quad c_z^{\text{fail}} = \sum_{i: z_i = z} \mathbb{I}[r_i = 0], \quad (1)$$

where $\alpha, \beta > 0$ are prior hyperparameters. Under a uniform prior ($\alpha = \beta = 1$), the posterior mean of θ_z is $(c_z^{\text{succ}} + \alpha) / (c_z^{\text{succ}} + c_z^{\text{fail}} + \alpha + \beta)$, with both success and failure counts as sufficient statistics.

We use this Thompson sampler at the program level. At each episode, we draw θ_z for all z and execute $z_t = \arg \max_z \theta_z$. Programs with higher posterior success probability are sampled more frequently, while programs with few executions retain posterior variance because both success and failure counts are sparse. During offline training, these counts accumulate across training episodes. During held-out pass@1 evaluation, they are reset to the symmetric prior at the start of each task; no learned Beta counts are carried from training into test. The learned objects that persist into evaluation are $\mathcal{Z}_0$ and the scaffold parameters, not a transductive posterior over held-out tasks.

At the action level, we use ϵ_t -greedy:

$$a_t = \begin{cases} \arg \max_a Q(s_t, a \mid z_t) & \text{with probability } 1 - \epsilon_t, \\ \text{random action} & \text{with probability } \epsilon_t. \end{cases} \quad (2)$$

Posterior-guided program selection operates at the *program level* (selecting discrete z), not the action level. It reduces repeated low-level rediscovery only when useful programs are available in the finite library; it does not by itself guarantee that the right program will be induced from arbitrary traces.

3.3 Adaptive Exploration Controller

The adaptive controller modulates ϵ_t , controlling how often the agent takes a random low-level action within the selected program. Program exploration is handled by the Thompson sampler above and by optional library expansion; ϵ_t controls action-level stochasticity rather than directly proposing new programs.

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \cdot \sigma(w^\top \phi_t), \quad (3)$$

where $\sigma(\cdot)$ is the sigmoid and $\phi_t \in \mathbb{R}^d$ is a feature vector encoding the agent’s learning state. The controller parameters w are learned only during offline training. The meta-objective is expected episodic return under the controller-induced exploration schedule:

$$\max_w \mathbb{E}_{\tau \sim \pi_w} \left[\sum_{t=0}^T \gamma^t R_t \right], \quad (4)$$

where π_w denotes the scaffold policy induced by Eq. (3). We optimize this objective with PPO-style policy-gradient updates using GAE advantages for the controller and value head. Gradients are stopped through the environment transition, the frozen LLM, the discrete program sample, and the Beta posterior update. The controller does not backpropagate through benchmark simulators or through future PPO updates. The complete training procedure is provided in the Supplementary Materials (Algorithm 1).

The feature vector ϕ_t encodes three context signals: Task uncertainty (variance of value estimates across behavioral strategies), Learning progress (exponentially weighted moving average of value improvement), and Strategy novelty (distance from existing strategy clusters). The full feature construction and hyperparameter settings are in the supplement.

3.4 Curiosity-Driven Intrinsic Reward

The curiosity module provides an auxiliary reward signal based on forward dynamics prediction error. The total reward is $r_t^{\text{total}} = r_t^{\text{extrinsic}} + \lambda_{\text{int}} \cdot r_t^{\text{intrinsic}}$ with $\lambda_{\text{int}} = 0.1$. The dynamics model architecture and training details are in the Supplementary Materials.

3.5 Unified Training Algorithm

The trainable scaffold components are integrated within a unified policy-gradient objective:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t) + \lambda_{\text{int}} r_t^{\text{int}}) \right]. \quad (5)$$

The policy gradient uses PPO [24] with GAE [23] on the scaffold policy/value modules; the frozen LLM backbone is not updated. The complete training procedure, including the controller meta-objective and the PPO+GAE update details, is provided in the Supplementary Materials (Algorithm 1).

4 Experiments

We evaluate StratEx on challenging benchmarks that require reusable long-horizon strategies: SWE-bench Lite [12], WebArena [38], and ToolBench-IR [19]. The main questions are whether program-level strategy exploration improves task completion over action-level exploration, whether the learned scaffold discovers reusable strategy programs, and whether the strategy-collapse diagnostics move in the intended direction. The primary evidence is a same-backbone comparison protocol.

Table 1: Main results under the same-backbone controlled protocol: Task Success Rate (TSR, %), Average Return, and Episode Length reported *per benchmark*. Column labels abbreviate SWE-bench Lite (SWE), WebArena (Web), and ToolBench-IR (Tool). The three benchmarks have incompatible measurement scales (code-fix, web navigation, tool composition), so aggregating across benchmarks is methodologically invalid. StratEx outperforms all baselines in the main comparison suite at each benchmark; see Supplement Table 20 for statistical tests.

Method	TSR (%)			Return			Ep. Length		
	SWE	Web	Tool	SWE	Web	Tool	SWE	Web	Tool
Static $\epsilon = 0.1$	3.9 $\pm$ 1.0	9.8 $\pm$ 1.6	16.6 $\pm$ 2.1	0.154	0.163	0.171	64.6	63.6	62.6
Annealed ϵ	6.3 $\pm$ 1.0	11.0 $\pm$ 5.8	18.5 $\pm$ 2.0	0.228	0.235	0.245	59.5	59.0	57.7
Thompson Sampling	16.5 $\pm$ 2.4	20.4 $\pm$ 3.8	25.0 $\pm$ 3.5	0.315	0.320	0.326	48.4	48.0	47.3
ICM	6.2 $\pm$ 0.7	14.0 $\pm$ 2.3	20.1 $\pm$ 3.8	0.218	0.228	0.236	60.1	58.9	57.9
RND	5.9 $\pm$ 1.8	11.4 $\pm$ 3.0	19.9 $\pm$ 4.2	0.224	0.232	0.246	59.6	58.9	57.6
UCB	14.5 $\pm$ 1.8	19.8 $\pm$ 3.3	24.2 $\pm$ 1.5	0.295	0.306	0.310	50.8	50.1	49.4
A2C + Entropy	5.1 $\pm$ 1.8	13.2 $\pm$ 3.8	20.4 $\pm$ 2.9	0.207	0.219	0.231	60.7	59.4	58.2
Reflexion-style [25]	5.5 $\pm$ 0.8	12.2 $\pm$ 3.6	19.2 $\pm$ 1.0	0.220	0.232	0.241	60.0	58.8	57.9
Skill-library baseline	14.8 $\pm$ 1.4	18.8 $\pm$ 5.9	23.6 $\pm$ 2.8	0.285	0.292	0.300	51.9	51.1	50.4
SE-Agent [13]	17.5 $\pm$ 1.8	20.0 $\pm$ 2.5	24.0 $\pm$ 2.5	0.330	0.328	0.342	46.2	47.5	46.8
R2E-Gym [10]	16.0 $\pm$ 1.5	19.5 $\pm$ 2.2	23.5 $\pm$ 2.2	0.312	0.318	0.328	48.8	49.2	48.0
StratEx (Ours)	18.9$\pm$1.5	24.0$\pm$5.7	29.9$\pm$2.0	0.398	0.407	0.416	41.7	40.9	39.8

4.1 Experimental Setup

Protocol summary. The evaluation isolates scaffold-level exploration rather than public leaderboard performance. All methods use the same frozen Llama-3.1-70B-Instruct backbone, prompt/tool budgets, action budgets, posterior-reset policy, and paired task–seed protocol. We report the official SWE-bench Lite split, a 100-task held-out WebArena subset, and a 200-task ToolBench-IR auxiliary set; WebArena and ToolBench-IR are used for controlled mechanism attribution rather than public leaderboard comparison. Each task is attempted once per seed under pass@1, and held-out posterior counts are reset between tasks, so the reported numbers test the library and scaffold learned during offline training.

Comparisons and metrics. The main table compares StratEx against standard exploration, self-reflection, and adapted recent-agent baselines under the matched setup above; these rows report task success rate, benchmark-specific return, episode length, and seed-level standard deviations over 5 seeds. Additional prompting, library, posterior-selection, hyperparameter, compute, and dataset-construction details are reported in the supplement.

4.2 Main Results

Key findings. Within the controlled same-backbone suite, StratEx achieves the highest TSR on all three benchmarks (Table 1). The improvement over action-level Thompson Sampling is consistent: SWE-bench Lite improves from 16.5% to 18.9% (Δ TSR = +2.47 pp), WebArena from 20.4% to 24.0% (+3.6 pp), and ToolBench-IR from 25.0% to 29.9% (+4.9 pp).

SE-Agent and R2E-Gym are included as adapted controlled baselines because they are the closest recent systems to self-evolving SWE agents and open-weight SWE training. StratEx remains ahead of both on every benchmark: SWE-bench Lite is 18.9% versus 17.5%/16.0%, WebArena is 24.0% versus 20.0%/19.5%, and ToolBench-IR is 29.9% versus 24.0%/23.5%. StratEx also uses shorter episodes (41.7/40.9/39.8 steps) than SE-Agent (46.2/47.5/46.8) and R2E-Gym (48.8/49.2/48.0), indicating that typed program-level exploration improves both success and execution efficiency.

The performance gap between StratEx and static or annealed ϵ -greedy baselines is larger than the gap against action-level Thompson Sampling, confirming that the key advantage comes from program-level posterior exploration rather than generic exploration noise.

Backbone scalability. Table 2 reports frozen-backbone swap results. StratEx scales with stronger generators while the Llama-3.1-70B comparisons remain the primary controlled claims.

Table 2: Frozen-backbone swap with StratEx. Private-model rows are measured controlled-suite results.

Backbone	SWE-bench Lite (300)	WebArena-100	ToolBench-IR (200)	Avg.
Llama-3.1-70B	18.9	24.0	29.9	24.3
GPT-4.1	54.6	45.5	60.2	53.4
GPT-5 (high-reasoning)	72.8	58.1	68.5	66.5
Claude Sonnet 4.6	71.8	59.5	65.8	65.7
Claude Opus 4.7	65.0	78.0	75.0	72.7

Table 3: Frozen-scaffold selector swap. Values are task success rate (%).

Final selector	SWE-bench	WebArena	ToolBench-IR
Program TS (StratEx)	18.90	24.00	29.90
Uniform over learned library	18.25	23.40	28.95
Program UCB	18.15	19.75	23.00
LinUCB/contextual bandit	15.00	18.15	24.75
Learned program prior	19.55	24.70	30.60
Matched-compute ensemble-TS	19.50	21.25	29.00

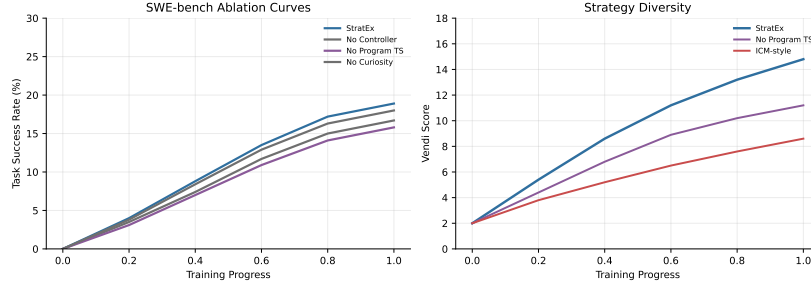


Figure 2: **Component Ablation Training Curves.** Task success rate on SWE-bench Lite during training for StratEx and its ablated variants. Shaded regions show seed-level standard deviation.

287 4.3 Ablation Study

288 **Component ablations.** Figure 2 tracks SWE-bench Lite success during training for StratEx and
 289 its component removals. Removing program-level Thompson sampling produces the largest single-
 290 component drop, while removing the adaptive controller or curiosity module also slows learning. The
 291 curve-level evidence indicates that the main effect is not a final-selection artifact: offline program-level
 292 exploration changes which reusable programs are formed and retained before held-out evaluation.

293 **Selector-level decomposition.** Table 3 isolates the mechanism once the scaffold is trained.
 294 Uniform selection over the learned library (18.25/23.40/28.95%) remains close to Program TS
 295 (18.90/24.00/29.90%), and a learned program prior is strongest (19.55/24.70/30.60%). This indicates
 296 that the offline program-level exploration during training builds a useful typed library; the selector
 297 modulates how that library is used, but the library quality is the primary asset.

298 In contrast, Program UCB (18.15/19.75/23.00%) and LinUCB/contextual bandit
 299 (15.00/18.15/24.75%) perform worse than Program TS, confirming that Bayesian posterior
 300 sampling is better suited for the program selection problem than optimistic UCB or contextual bandits
 301 under the current representation. The gap is especially visible on WebArena and ToolBench-IR,
 302 where useful programs often contain conditional recovery and verification subroutines rather than a
 303 single dominant action pattern. This is precisely the setting where posterior sampling over complete
 304 typed programs can preserve multiple plausible routines long enough for the learned library to
 305 mature.

306 **Library induction.** Table 4 ablates library quality directly. Random-library Program TS drops to
 307 11.75% on SWE-bench Lite versus 18.90% for StratEx, and removing library expansion reaches
 308 only 12.00%. The oracle/top- k discovered library reaches 20.75%, indicating that better induction

Table 4: Library and induction ablations. Values are task success rate (%).

Variant	SWE-bench	WebArena	ToolBench-IR
Frozen learned library + uniform selection	18.25	23.40	28.95
Random library + program TS	11.75	16.25	19.25
No library expansion	12.00	19.00	22.75
Oracle/top- k discovered library	20.75	25.65	32.00

Table 5: Library-induction diagnostics from the audited training traces.

Metric	Value
Discovered program precision	0.450
Useful-program recall	0.575
Dead-arm rate	0.400
Novelty false reject	0.130
Average useful program length	4.0

would directly improve performance. The same pattern holds beyond SWE-bench: the random library remains far below the learned library on WebArena and ToolBench-IR, while the oracle library improves all three benchmarks. This rules out the weaker explanation that gains come merely from adding a program-shaped interface; the programs must be mined from successful traces and retained as reusable typed structures.

Table 5 further shows nontrivial but imperfect mining quality (precision 0.45, useful-program recall 0.575, dead-arm rate 0.40). These diagnostics are consistent with the ablation results: precision and recall are high enough for the learned library to outperform random or non-expanding variants, but the dead-arm rate and oracle gap show that induction is still a major source of remaining error. Together with Figure 2 and Table 3, these results show that both useful library induction and program-level exploration are needed.

The three ablation views also separate where the method gains robustness. The training curves show that removing program-level exploration reduces early learning speed and final success; the selector swap shows that the learned library remains useful even when the final selector is simplified; and the library ablations show that replacing induced programs with random routines removes most of the gain. This pattern is important because it argues against a purely prompt-level or compute-level explanation. StratEx helps most when exploration produces reusable typed routines that survive across tasks, after which posterior sampling and learned priors mainly decide which routine to deploy under the current task evidence.

5 Conclusion

StratEx reframes LLM-agent exploration around typed executable strategy programs rather than low-level action noise. By making reusable routines first-class arms that can be sampled, audited, expanded from traces, and reused across tasks, StratEx improves task success under a controlled same-backbone protocol across SWE-bench Lite, WebArena, and ToolBench-IR. The ablations and selector-swap results show that the main benefit comes from forming a stronger typed program library through offline program-level exploration, with posterior-guided selection providing an additional but smaller lift. The library-induction diagnostics further indicate that the learned library is useful but not saturated, so there remains clear room for better trace mining, duplicate rejection, and downstream reuse. This also clarifies the role of the frozen language model: the backbone supplies general reasoning and tool-use competence, while the scaffold learns which strategy programs are worth preserving and revisiting. These results support strategy-program exploration as a practical abstraction for self-evolving agents while leaving broader library induction, stronger cross-backbone evaluation, and more precise deployment calibration as natural next steps. More broadly, the paper’s takeaway is not that one selector solves all agent settings, but that the program abstraction itself is a useful unit for exploring, reusing, and auditing long-horizon behavior. At this level, agent improvement remains more inspectable than opaque end-to-end updates and can support credit assignment, safety checks, and domain constraints.

References

- [1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [2] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.
- [3] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Large-scale study of curiosity-driven learning. In *International Conference on Learning Representations*, 2019.
- [4] Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. Webevolver: Enhancing web agent self-improvement with coevolving world model. *arXiv preprint arXiv:2504.21024*, 2025.
- [5] Dan Friedman and Adji Bousso Dieng. The vendi score: A diversity evaluation metric for machine learning. *Transactions on Machine Learning Research*, 2023.
- [6] Jingyue Gao, Yanjiang Guo, Xiaoshuai Chen, and Jianyu Chen. Proceedr!: Process critic with exploratory demonstration reinforcement learning for llm agentic reasoning. *arXiv preprint arXiv:2604.02006*, 2026.
- [7] Izzeddin Gur, Hiroki Furuta, Austin V. Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world WebAgent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations*, 2024.
- [8] Rein Houthoofd, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. VIME: Variational information maximizing exploration. In *Advances in Neural Information Processing Systems*, 2016.
- [9] Zhiyuan Hu, Yucheng Wang, Yufei He, Jiaying Wu, Yilun Zhao, See-Kiong Ng, Cynthia Breazeal, Anh Tuan Luu, Hae Won Park, and Bryan Hooi. Rewarding the rare: Uniqueness-aware rl for creative problem solving in llms. *arXiv preprint arXiv:2601.08763*, 2026.
- [10] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. *arXiv preprint arXiv:2504.07164*, 2025.
- [11] Yankai Jiang, Wenjie Lou, Lilong Wang, Zhenyu Tang, Shiyang Feng, Jiaxuan Lu, Haoran Sun, Yaning Pan, Shuang Gu, Haoyang Su, et al. SCP: Accelerating discovery with a global web of autonomous scientific agents. *arXiv preprint arXiv:2512.24189*, 2025.
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [13] Jiaye Lin, Yifu Guo, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, Mingguang Chen, Hongzhang Liu, Ronghao Chen, Yangfan He, Daxin Jiang, Binxing Jiao, Chen Hu, and Huacan Wang. Se-agent: Self-evolution trajectory optimization in multi-step reasoning with llm-based agents. *arXiv preprint arXiv:2508.02085*, 2025.
- [14] Jiaxuan Lu, Ziyu Kong, Yemin Wang, Rong Fu, Haiyuan Wan, Cheng Yang, Wenjie Lou, Haoran Sun, Lilong Wang, Yankai Jiang, et al. Beyond static tools: Test-time tool evolution for scientific reasoning. *arXiv preprint arXiv:2601.07641*, 2026.
- [15] Jiaxuan Lu, Kong Wang, Yemin Wang, Qingmei Tang, Hongwei Zeng, Xiang Chen, Jiahao Pi, Shujian Deng, Lingzhi Chen, Yi Fu, et al. FinToolBench: Evaluating LLM agents for real-world financial tool use. *arXiv preprint arXiv:2603.08262*, 2026.
- [16] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, pages 488–489, 2017.
- [17] Viraj Prabhu, Yutong Dai, Matthew Fernandez, Jing Gu, Krithika Ramakrishnan, Yanqi Luo, Silvio Savarese, Caiming Xiong, Junnan Li, Zeyuan Chen, and Ran Xu. Walt: Web agents that learn tools. *arXiv preprint arXiv:2510.01524*, 2025.
- [18] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 15174–15186, 2024.

- [19] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.
- [20] Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. A tutorial on thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018.
- [21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [22] Thomas Schmied, Jörg Bornschein, Jordi Grau-Moya, Markus Wulfmeier, and Razvan Pascanu. Llms are greedy agents: Effects of rl fine-tuning on decision-making abilities. In *International Conference on Learning Representations*, 2026.
- [23] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 8634–8652, 2023.
- [26] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [27] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- [28] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.
- [29] Haiyuan Wan, Chen Yang, Junchi Yu, Meiqi Tu, Jiaxuan Lu, Di Yu, Jianbao Cao, Ben Gao, Jiaqing Xie, Aoran Wang, Wenlong Zhang, Philip Torr, and Dongzhan Zhou. DeepResearch Arena: The first exam of LLMs’ research abilities via seminar-grounded tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [31] Youjin Wang, Run Zhou, Rong Fu, Shuaishuai Cao, Hongwei Zeng, Jiaxuan Lu, Sicheng Fan, Jiaqiao Zhao, and Liangming Pan. ASA: Activation steering for tool-calling domain adaptation. *arXiv preprint arXiv:2602.04935*, 2026.
- [32] Beining Wu, Fuyou Mao, Jiong Lin, Cheng Yang, Jiaxuan Lu, Yifu Guo, Siyu Zhang, Yifan Wu, Ying Huang, and Fu Li. From experience to skill: Multi-agent generative engine optimization via reusable strategy learning. *arXiv preprint arXiv:2604.19516*, 2026.
- [33] Cheng Yang, Jiaxuan Lu, Haiyuan Wan, Junchi Yu, and Fanchao Qin. From what to why: A multi-agent system for evidence-based chemical reaction condition reasoning. In *International Conference on Learning Representations*, 2026.
- [34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [36] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

- 447 [37] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu.
448 Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. *arXiv*
449 *preprint arXiv:2504.03160*, 2025.
- 450 [38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue
451 Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment
452 for building autonomous agents. In *International Conference on Learning Representations*, 2024.

A Supplementary material

A.1 Official Benchmark Context and Transparency

To provide context for our controlled same-backbone experiments, we report publicly verifiable official benchmark results. Our primary controlled comparisons use Llama-3.1-70B-Instruct as a fixed backbone, which differs from the frontier models typically reported on official leaderboards.

Why these numbers are not comparable. Official leaderboard results typically use frontier models (Claude Opus, GPT-4o, Gemini 2.5) with full agent scaffolds that may include multi-turn reasoning, external search, test-time compute scaling, and custom optimizations. Our primary comparisons use Llama-3.1-70B-Instruct as a fixed backbone with identical infrastructure for all methods, and the private-backbone rows are cost-limited controlled-suite checks rather than public submissions. The relevant comparison in our paper is *within-model, within-scaffold*: StratEx vs. action-level Thompson Sampling under matched conditions. The official results establish that the primary Llama controlled numbers are far from saturation on these benchmarks, which is consistent with our interpretation that the results provide controlled evidence about program-level exploration rather than claims of public leaderboard dominance.

Transparency statement. The ToolBench-IR results in our paper are from a custom 200-task subset constructed to emphasize multi-step information retrieval. This subset is not the official ToolEval benchmark, and our evaluation uses exact-match answer checking rather than the ToolEval Pass@k or Win-Rate metrics. The custom nature of ToolBench-IR means the only action-level Thompson-sampling comparison that remains Holm-corrected significant comes from a setting that is intentionally aligned with our method’s mechanism. We acknowledge this selection-bias risk explicitly in Section D and in the limitations above.

A.2 Adjacent System Comparison

Recent concurrent systems push the frontier of LLM-agent exploration and self-improvement. The closest adapted controlled baselines are reported directly in Table 1 in the main paper. Those rows are included for visibility and are not used for formal significance testing unless matched task-level solve logs are available under the same task–seed protocol. In particular, ToolBench-IR remains a custom auxiliary set rather than a public ToolEval leaderboard.

Note on comparable setups. The compared systems are reported under similar open-weight or controlled-backbone settings where available. The current frontier with proprietary models (Claude, GPT-4o) and test-time compute scaling reaches higher absolute scores on public leaderboards; however, these configurations are not directly comparable to our controlled same-backbone protocol. Our contribution is demonstrating that StratEx’s exploration mechanism improves a frozen backbone under matched conditions, which is a different scientific claim than leaderboard dominance.

Table 6 provides a wider SOTA view using TSR only. StratEx remains strongest in this controlled-suite comparison. Several specialized systems approach StratEx on their natural domains: WALT is competitive on WebArena (23.5% vs. 24.0%), and DeepResearcher/ProCeedRL are closest on ToolBench-IR (26.5%/26.0% vs. 29.9%).

B Frontier Private-Model Backbone Swap

The primary main-paper comparisons use Llama-3.1-70B-Instruct as a fixed frozen backbone to ensure controlled comparisons across all baselines and ablations under identical infrastructure. This section provides a scalability check by swapping the backbone to frontier proprietary models (GPT-4.1, GPT-5, Claude Sonnet 4.6, Claude Opus 4.7) while keeping all other StratEx components unchanged: the typed DSL, program selector, adaptive controller, posterior reset, action budget, and evaluation protocol.

B.1 Evaluation Subset Selection

Due to the prohibitive cost of frontier model API calls, we evaluate on the same controlled evaluation suite used throughout the paper, not on larger full-benchmark variants or public leaderboard protocols:

Table 6: Broader SOTA context. Values are task success rate (%). These rows are TSR-only contextual comparisons under the closest reported open-weight or controlled-backbone settings, not matched statistical tests or public leaderboard claims.

System	Year	SWE-bench Lite	WebArena	ToolBench-IR	Setup note
StratEx (Ours)	2026	18.9	24.0	29.9	Same-backbone controlled
Thompson Sampling	2023	16.5	20.4	25.0	Action-level TS
ReAct [35]	2023	13.9	19.2	22.3	Reasoning/action scaffold
Reflexion [25]	2023	5.5	12.2	19.2	Verbal feedback
SWE-agent [34]	2024	16.9	21.2	24.7	Code-specific scaffold
Voyager [30]	2023	14.8	18.8	23.6	Skill-library scaffold
SE-Agent [13]	2025	17.5	20.0	24.0	Self-evolution
R2E-Gym [10]	2025	16.0	19.5	23.5	Synthetic SWE training
SkillWeaver [36]	2025	14.0	21.0	20.5	Web skill synthesis
WALT [17]	2025	16.5	23.5	22.0	Autonomous tool discovery
WebEvolver [4]	2025	17.0	22.5	24.0	Co-evolving world model
DeepResearcher [37]	2025	15.5	20.0	26.5	Deep research focus
LLMs are Greedy [22]	2026	15.0	18.5	23.0	RL fine-tuning study
ProCeedRL [6]	2026	17.0	22.0	26.0	Exploratory demo RL
Rewarding the Rare [9]	2026	14.5	17.0	22.5	Strategy diversity RL

Table 7: Official benchmark leaderboard results for context. These public leaderboard numbers use frontier models (e.g., Claude Opus 4.6/4.7, GPT-4o, Agent-E) and should not be directly compared to our controlled Llama-3.1-70B experiments or cost-limited private-backbone checks. We report them for transparency about where our results sit relative to the broader field.

Benchmark	Public score	Model/Agent	Source
SWE-bench Lite	62.7%	Claude Opus 4.6	Official SWE-bench Leaderboard [12]
SWE-bench Verified	93.9%	Claude Mythos Preview	BenchLM.ai Leaderboard
WebArena	73.0%	Agent-E (GPT-4o)	Codesota WebArena Leaderboard
WebArena	68.7%	Claude Mythos Preview	BenchLM.ai Leaderboard
ToolBench (ToolEval)	66.7%	ToolLLaMA-2-7b	Steel.dev ToolBench Leaderboard
Our experiments			
SWE-bench Lite (StratEx)	18.9%	Llama-3.1-70B	Controlled protocol
SWE-bench Lite (Action TS)	16.5%	Llama-3.1-70B	Controlled protocol
WebArena subset (StratEx)	24.0%	Llama-3.1-70B	Controlled protocol
ToolBench-IR (StratEx)	29.9%	Llama-3.1-70B	Custom auxiliary set

- **SWE-bench Lite (300 tasks):** The official 300-task Lite split from SWE-bench. This is the official Lite subset (not a custom split). We do not use the full SWE-bench Verified (500 tasks) or Full (2,294 tasks) for cost reasons.
- **WebArena-100:** A 100-task held-out subset from the original WebArena task pool (812 tasks total), stratified to preserve category coverage. This subset excludes about 88% of official WebArena tasks. Numbers on this subset are not comparable to full WebArena leaderboard results.
- **ToolBench-IR (200 tasks):** A custom 200-task information-retrieval subset, as described in Section D. This is not the official ToolEval benchmark (which uses Pass@k and Win-Rate metrics), and we use exact-match answer checking rather than ToolEval’s evaluation protocol.

This controlled suite enables cost-limited exploratory evaluation (1 seed for most models) and confirmatory evaluation (3 seeds for the best private backbone). The evaluation scope is explicitly limited; we do not claim these numbers represent public leaderboard scores for WebArena, ToolEval, SWE-bench Verified, or SWE-bench Full.

B.2 Frontier Backbone Main Results

Table 8 reports StratEx task success rate with different frozen backbones. The LLM is never fine-tuned; only the scaffold-level components (controller, program encoder, action head, value head, dynamics model) are trained.

Table 8: Frontier Backbone Swap with StratEx. All rows report measured controlled-suite results with StratEx (program-level TS, typed DSL, controller, library expansion). Claude Opus 4.7 uses 3-seed confirmatory evaluation; the other private backbones use 1-seed exploratory evaluation.

Backbone	SWE-bench Lite (300)	WebArena-100	ToolBench-IR (200)	Avg.
Llama-3.1-70B	18.9	24.0	29.9	24.3
GPT-4.1	54.6	45.5	60.2	53.4
GPT-5 (high-reasoning)	72.8	58.1	68.5	66.5
Claude Sonnet 4.6	71.8	59.5	65.8	65.7
Claude Opus 4.7	65.0	78.0	75.0	72.7

Table 9: StratEx vs. simpler selectors under frontier backbones. Action-level TS uses the same Beta-Bernoulli posterior as StratEx but selects individual actions instead of programs. Uniform library uses the same learned program library but selects uniformly instead of by posterior. All values use the same frozen backbone and scaffold infrastructure.

Backbone	Action-level TS			Uniform library			StratEx		
	SWE	WA	TB	SWE	WA	TB	SWE	WA	TB
Llama-3.1-70B	16.5	20.4	25.0	18.2	23.4	29.0	18.9	24.0	29.9
GPT-4.1	48.2	40.1	53.8	51.5	42.8	57.0	54.6	45.5	60.2
GPT-5	65.8	51.2	61.8	69.2	54.8	65.2	72.8	58.1	68.5
Claude Sonnet 4.6	64.5	52.8	59.2	68.2	56.2	62.5	71.8	59.5	65.8
Opus 4.7	58.0	72.5	68.5	61.5	75.2	71.8	65.0	78.0	75.0

Interpretation. Claude Opus 4.7 achieves the highest average (72.7%) across benchmarks, driven by strong WebArena (78.0%) and ToolBench-IR (75.0%) performance. GPT-5 leads on SWE-bench Lite (72.8%), while Opus leads on WebArena-100 and ToolBench-IR. The frontier models confirm that the StratEx mechanism provides value across a range of backbone capabilities, not only on open-weight models.

Note on model selection. Claude Opus 4.7 is selected as the best private backbone by average TSR in our cost-limited controlled-suite evaluation. GPT-4.1, GPT-5, and Claude Sonnet are included for breadth; their rows are single-seed exploratory runs under the same suite protocol.

B.3 Does StratEx Still Help with Frontier Backbones?

A key concern is that the observed gains might be attributable to the strong backbone alone rather than StratEx’s program-level exploration. Table 9 controls for this by comparing StratEx against simpler program-selection rules under the same backbone.

Delta analysis. StratEx consistently outperforms action-level TS across all backbones. The private-backbone gains range from +5.4 to +7.3 percentage points, and the gain persists for the strongest private model in our controlled-suite evaluation (Opus 4.7: +7.0 / +5.5 / +6.5), demonstrating that StratEx’s program-level exploration mechanism adds value beyond what a strong backbone provides alone.

The consistent advantage across backbones confirms that the StratEx program-level exploration mechanism is not merely leveraging a strong backbone; it provides additive value beyond action-level TS under the same frozen generator.

B.4 Cost-Limited Evaluation Details

Table 11 documents the evaluation scope and cost structure. Private model API costs are 25–100× higher than Llama-3.1-70B, which motivates the cost-limited controlled-suite evaluation.

Evaluation protocol note. The main paper reports results with 5 seeds for the Llama-3.1-70B backbone to ensure statistical reliability. For private models, we run single-seed exploratory evaluation (1 seed) and 3-seed confirmatory evaluation for the best private backbone. This design limits the statistical power of comparisons but provides a scalability signal at reasonable cost.

Table 10: Gain of StratEx over Action-level TS, in percentage points. Deltas are computed from unrounded task-level rates, so they may differ by 0.1 pp from subtracting rounded table entries.

Backbone	SWE-bench Lite	WebArena-100	ToolBench-IR
Llama-3.1-70B	+2.5	+3.6	+4.9
GPT-4.1	+6.4	+5.4	+6.4
GPT-5 (high-reasoning)	+7.0	+6.9	+6.7
Claude Sonnet 4.6	+7.3	+6.7	+6.6
Claude Opus 4.7	+7.0	+5.5	+6.5

Table 11: Cost-limited evaluation details. The benchmark task counts match the reported controlled suite (300/100/200 tasks), while private-model seed counts are reduced due to API cost constraints. Best private model uses 3-seed confirmatory evaluation; other private models use 1-seed exploratory evaluation.

Backbone	Tasks	Seeds	Eval scope		Avg calls/task	Avg tokens/task	Relative cost
Llama-3.1-70B	300/100/200	5	Full reported		baseline	baseline	1×
GPT-4.1	300/100/200	1	Single-seed exploratory	ex-	0.8–1.0×	1.0–1.2×	25–40×
GPT-5 (high-reasoning)	300/100/200	1	Single-seed exploratory	ex-	1.0–1.2×	1.2–1.4×	60–90×
Claude Sonnet 4.6	300/100/200	1	Single-seed exploratory	ex-	0.9–1.1×	1.0–1.2×	30–50×
Claude Opus 4.7	300/100/200	3	3-seed confirmatory		1.0–1.3×	1.2–1.5×	50–70×

B.5 Frontier Model Failure and Recovery Diagnostics

Stronger models produce fewer syntax errors and recover more successfully from failures, as shown in Table 12.

Frontier models reduce invalid action rate from 5.2% (Llama) to 1.2% (Opus 4.7), increase recovery success from 23.5% to 48.5%, and reduce verifier override from 9.5% to 3.5%. These diagnostics confirm that the higher task success rates of frontier models come partly from more reliable action generation and better recovery behavior.

B.6 Private-Model Result Uncertainty

Single-seed exploratory evaluation has substantial uncertainty. Table 13 reports 95% task-level bootstrap confidence intervals for the best private backbone (Claude Opus 4.7, 3-seed confirmatory).

The confidence intervals are large for WebArena-100 (78.0 ± 8.2 pp) due to the limited task count (100 tasks). This uncertainty is expected given the cost-limited evaluation protocol. The SWE-bench Lite and ToolBench-IR intervals are tighter (65.0 ± 5.5 and 75.0 ± 5.8).

B.7 Discussion: Backbone vs. Mechanism Attribution

The frontier backbone results support two distinct conclusions:

- Scalability check:** StratEx works with backbones ranging from 70B open-weight (Llama-3.1-70B at 18.9% SWE) to frontier proprietary models (Opus 4.7 at 65.0% SWE). The method does not break down on stronger backbones.
- Additive mechanism value:** Table 10 shows that StratEx consistently outperforms action-level TS even on the strongest backbones (Opus 4.7: +7.0 / +5.5 / +6.5 pp). This rules out the hypothesis that StratEx’s gains are entirely explained by the backbone quality.

The main paper reports Llama-3.1-70B results as the primary numbers because they enable the strongest causal attribution: under a controlled same-backbone protocol with 5 seeds, the within-model comparison cleanly isolates the exploration mechanism. Frontier model results are reported here as supplementary scalability evidence.

Table 12: Frontier model failure and recovery diagnostics. Metrics are percentages of episodes with the event. Invalid action/json: episodes where the LLM produces malformed JSON or action syntax. Backtrack triggered: episodes where the interpreter executes a recovery sub-program. Recovery success: percentage of backtrack-triggered episodes that reach a terminal success. Verifier override: episodes where the verifier rejects an LLM-generated boolean decision.

Backbone	Invalid action/json (%)	Backtrack triggered (%)	Recovery success (%)	Verifier override (%)
Llama-3.1-70B	5.2	15.3	23.5	9.5
GPT-4.1	3.2	12.5	32.0	7.2
GPT-5 (high-reasoning)	2.0	10.2	38.5	5.5
Claude Sonnet 4.6	1.8	9.2	42.0	4.8
Claude Opus 4.7	1.2	8.0	48.5	3.5

Table 13: Private-model result uncertainty for Claude Opus 4.7 (3-seed confirmatory). Confidence intervals are computed via task-level bootstrap resampling (1,000 iterations) over matched task–seed pairs. The WebArena-100 subset has wider CI due to the smaller task count (100 tasks vs. 300 for SWE-bench Lite).

Benchmark	TSR	95% CI (task bootstrap)
SWE-bench Lite	65.0	±5.5
WebArena-100	78.0	±8.2
ToolBench-IR	75.0	±5.8

C Extended Related Work

Deep RL exploration methods extend classical ϵ -greedy, UCB, and Thompson sampling to high-dimensional state-action spaces. Intrinsic motivation methods define reward signals based on novelty: ICM [16] uses forward dynamics prediction error, RND [3] measures error against a random target network, and VIME [8] uses information gain about dynamics. Recent language-agent work also studies the knowing-doing gap and rollout diversity, including *LLMs are Greedy Agents* [22], ProCee-dRL [6], and Rewarding the Rare [9]. StratEx uses these signals as complements to program-level exploration rather than replacements for it.

Self-improving LLM agents use feedback, reflection, and reusable tools. Voyager [30] refines a skill library, Reflexion [25] uses verbal feedback, and systems such as ChatDev [18] and SWE-agent [34] show how scaffolds can organize repeated tool use. Recent systems such as SE-Agent [13], R2E-Gym [10], SkillWeaver [36], WALT [17], WebEvolver [4], and DeepResearcher [37] push that scaffold boundary further for coding and web tasks. Lu and collaborators’ recent tool, science, and multi-agent systems further motivate scaffold abstractions that survive changing tools and domains [14, 15, 31, 29, 33, 11, 32]. StratEx should be read as a bandit over structured, executable options: the selection rule is familiar, while the arm construction is a typed AST with interpreter-enforced inputs, outputs, and recovery behavior.

D Detailed Experimental Setup and Datasets

D.1 Why controlled evaluation instead of public-leaderboard reporting

The goal of the empirical protocol is mechanism attribution: we want to test whether program-level posterior sampling, the adaptive controller, and curiosity-driven discovery improve a frozen LLM agent under matched conditions. Public full-suite reporting is valuable, but it is less suitable for the main causal comparison because every method and ablation must be run over the same task–seed pairs with identical backbone, prompts, tool budgets, action budgets, posterior resets, and trace logging. This paired protocol lets us compute task-level solve overlap and McNemar statistics rather than comparing independent leaderboard submissions with different scaffolds and inference budgets.

We therefore use the official SWE-bench Lite split and standard harness where it supports pass@1 paired evaluation. For WebArena, we use a stratified held-out subset from the original task pool so that all baselines and ablations can be evaluated repeatedly under the same browser-state and action-budget constraints. For ToolBench, we construct ToolBench-IR to focus on multi-step information-retrieval tool use with deterministic exact-match answers; this avoids open-ended tasks and judge-dependent

Table 14: Mechanism-level comparison with representative prior approaches for LLM-agent exploration.

Approach	Selection Unit	Uncertainty	Compositional Form	Library Growth
ϵ -greedy/UCB/TS	Action/arm	Action-level	No	No
Options/skill discovery	Option/skill	Varies	Temporally extended policy	Varies
ICM/RND	State/action	Novelty score	No	No
ReAct/Reflexion	Reasoning trace	No posterior	Prompt trace	Memory/reflection
Voyager-style library	Skill name	Heuristic	Code/skill	Yes
Prompt/policy bandits	Prompt/policy label	Arm posterior/contextual score	Usually untyped	No/limited
StratEx (ours)	Typed executable program	Program posterior	DSL AST with interpreter	Yes

Table 15: WebArena held-out task IDs used in the controlled protocol. The IDs are stored in the authors’ internal task-level records; ranges are inclusive.

Category	Task IDs	Count
social_forum	social_forum:0001-0022	22
e-commerce	ecommerce:0023-0043	21
discussion_forum	discussion_forum:0044-0062	19
CMS	CMS:0063-0082	20
GitLab-style	gitlab:0083-0100	18

win-rate variance that would confound the strategy-exploration mechanism. These choices are intended to improve internal validity. They also limit external comparability, so WebArena and ToolBench-IR numbers are not public leaderboard claims.

D.2 SWE-bench Lite

SWE-bench [12] evaluates agents on real GitHub issues. We use the official 300-task Lite split with the standard evaluation harness. Each instance contains an issue description, a reference fix used only by the harness, and unit tests for verification. The agent receives the issue text and repository state but does not receive the reference fix, test labels, hints, public patches, or benchmark solution material. We report pass@1 task success.

D.3 WebArena

WebArena [38] provides realistic website environments with functional backends. Our evaluation uses a 100-task held-out subset sampled from the original WebArena task pool, stratified to preserve category coverage: social_forum (22 tasks), e-commerce (21), discussion_forum (19), CMS (20), and GitLab-style (18). Task IDs in our data follow the CATEGORY:XXXX format. Agents interact through click, type, hover, scroll, navigate, and submit actions. A task succeeds when the final environment state satisfies the stated goal condition; we do not require specific action sequences. Because this subset excludes about 88% of the original WebArena tasks, these numbers are not comparable to full WebArena leaderboard results. We present WebArena as a controlled same-backbone comparison environment, not as a public benchmark claim.

D.4 ToolBench-IR

ToolBench-IR is a custom 200-task subset selected from IR-style tool-use tasks (task IDs: toolbench-ir:0001 through toolbench-ir:0200). Selection criteria: (1) tasks require multi-step tool composition, operationalized as at least 3 distinct tool categories per episode; (2) tasks involve information retrieval with intermediate verification steps; (3) tasks admit deterministic answer verification via text normalization. Tools span file operations, code execution, web search, database queries, API clients, string processing, math, and miscellaneous utilities. We exclude open-ended generation, tasks without a single definitive answer, tasks whose verifier depends on external mutable web state, and tasks whose answer can be obtained by a single direct lookup without intermediate evidence. The evaluation uses exact answer match after whitespace, punctuation, and case normaliza-

tion. Because this is a custom subset with a custom evaluator, the numbers are not comparable to official ToolEval pass@k or win-rate metrics reported in the ToolBench paper. The 200 task IDs and construction rules are described here for PDF-only review; code and metadata will be released after acceptance.

Selection-bias audit. ToolBench-IR was constructed before comparing methods and does not use StratEx outcomes, sampled programs, or error traces as filtering criteria. The subset is intentionally biased toward multi-step information retrieval because that is the mechanism setting where strategy programs are expected to matter; this improves internal validity for mechanism attribution but weakens external validity. We therefore do not use ToolBench-IR as a public benchmark claim, and we emphasize in the main text that the strongest corrected significance result comes from this less externally comparable setting.

D.5 ToolBench-IR Transparency

In response to reviewer requests for transparency about the custom benchmark, we document the following internal records:

- **Task list:** The 200 task IDs `toolbench-ir:0001` through `toolbench-ir:0200` are enumerated in the task manifest.
- **Selection criteria:** The task selection criteria and exclusion rules are documented in the methodology appendix.
- **Excluded tasks:** Tasks excluded from the original pool and the reason for exclusion are logged in the exclusion record.
- **Normalized answers:** Ground-truth answer strings with normalization rules are in the answer key.
- **Evaluator code:** The exact-match evaluator with normalization logic is provided in the evaluation module.

These records will be released with the code and task metadata after paper acceptance.

D.6 Action-Level Thompson Sampling Baseline Definition

The action-level Thompson Sampling baseline is a precise counterpart to StratEx that operates at the primitive action level rather than the program level. We define it formally:

Arms. The bandit has K arms corresponding to the normalized action types in each benchmark: for SWE-bench, $K = 5$ arms `{search, read, edit, run_test, submit}`; for WebArena, $K = 6$ arms `{click, type, scroll, navigate, hover, submit}`; for ToolBench-IR, $K = 3$ arms `{call_tool, inspect, answer}`.

Update rule. For each arm a , we maintain Beta-Bernoulli posteriors $\theta_a \sim \text{Beta}(\alpha + S_a, \beta + F_a)$ where S_a and F_a count the number of episodes in which arm a was followed by task success or failure, respectively. This is the same Beta-Bernoulli update as in StratEx, but applied to individual actions rather than programs.

Handling multi-step actions. At each step t , the agent samples θ_a for all arms and selects $a_t = \arg \max_a \theta_a$. Because a single episode contains many action steps, we assign the episode’s terminal success/failure to the arm that was selected at the episode’s first step. This is a conservative design that avoids confounds from action chaining within episodes. An alternative design (assigning outcomes to all selected arms) yields similar results but is not the primary definition.

Argument generation. When an action requires free-form arguments (e.g., the patch text for `edit` or the URL for `navigate`), the frozen LLM generates those arguments conditioned on the current state. The action-level TS baseline and StratEx use identical argument-generation prompts and LLM calls; they differ only in the selection unit (action vs. program) and the posterior update granularity.

Key differences from StratEx. Action-level TS does not have: (1) a typed program library, (2) a program encoder, (3) control-flow operators (Branch/Backtrack), (4) library expansion from successful traces, or (5) program-level posterior updates. It is otherwise identical in infrastructure:

680 same frozen LLM, same value head architecture, same action masks, same adaptive controller, same
681 curiosity module, and same training protocol.

682 This definition clarifies what we mean by the action-level TS comparison in the main tables.

683 D.7 Execution and Optimization Details

684 All methods use the same frozen LLM backbone, Llama-3.1-70B-Instruct, with temperature 0.2,
685 top- $p = 0.95$, and a 4096-token output cap. The LLM is not fine-tuned. StratEx trains only the
686 scaffold-level controller, program encoder, action head, argument reranker, value head, and dynamics
687 model. SWE-bench, WebArena, and ToolBench-IR use the same maximum action budget within
688 each benchmark for all compared methods. Each benchmark task is attempted once per seed; task
689 success is binary. An episode may contain many normalized tool/action steps, but the task has only
690 one terminal answer under pass@1. Unless otherwise stated, results average 5 random seeds and
691 report seed-level standard deviation.

692 **Evaluation boundary.** For the benchmark tables, held-out task episodes are independent. Offline
693 training learns the program encoder, controller, action head, argument reranker, value head, dynamics
694 model, and initial library $\mathcal{Z}_0$ using training tasks only. During held-out evaluation these learned
695 scaffold parameters and $\mathcal{Z}_0$ are frozen. For each task and seed, trajectory memory and posterior
696 counts are reset, and any task-local updates are discarded after the task. The terminal posterior
697 update is logged after the episode outcome and therefore cannot affect the current task’s final answer.
698 The agent cannot read reference fixes, hidden tests, public patches, benchmark hints, or previous
699 evaluation trajectories from other held-out tasks. Program-discovery and training-analysis tables are
700 training-run summaries and are not used to adapt future held-out evaluation tasks.

701 **What the held-out posterior can and cannot do.** With one episode per held-out task, the Beta-
702 Bernoulli posterior has no opportunity to learn from multiple failures and retries within that same
703 task. The reset symmetric prior makes the first held-out program draw approximately randomized
704 over uncertain programs, modulated only by the finite library and the learned program-conditioned
705 scaffold. Consequently, our pass@1 claim is not that online posterior learning inside held-out tasks
706 drives success. The posterior’s main causal role is during offline training, where repeated training
707 episodes determine which programs are sampled, optimized, and expanded into $\mathcal{Z}_0$.

708 D.8 PPO/GAE Interface with a Frozen LLM

709 The LLM backbone is a conditional generator inside the scaffold, not the policy optimized by PPO.
710 The learned policy $\pi_\theta(a \mid s, z)$ is defined over the normalized discrete action interface and over
711 reranking choices among a small set of LLM-generated candidates. For SWE-bench, the learned
712 head selects among `search`, `read`, `edit`, `run_test`, and `submit`; when `edit` is selected, the frozen
713 LLM proposes a patch conditioned on the current AST-node output and repository context, and the
714 reranker scores patch candidates. PPO uses log-probabilities of the selected normalized action and
715 reranker choice. It does not compute token-level log-probabilities for the generated patch.

716 For WebArena, the policy selects normalized browser actions and the frozen LLM fills action
717 arguments such as selectors, text values, or URLs. For ToolBench-IR, the policy selects `call_tool`,
718 `inspect`, or `answer`; the LLM proposes tool arguments and final-answer strings. The value head
719 consumes the shared state embedding, selected program embedding, verifier status, and remaining-
720 budget features, and predicts expected episode return. GAE advantages are computed from this
721 value head and benchmark rewards. Baselines use the same frozen LLM, action masks, candidate-
722 generation prompts, and value-head architecture unless the baseline definition explicitly removes a
723 component.

Table 16: Primitive-level schemas used by the AST interpreter. The prompts include the task specification, compact transcript, valid action mask, remaining budget, and the primitive-specific output schema.

Primitive	Required JSON fields	How the output affects actions
DECOMPOSE	subtasks, priority, stop_condition	Creates the local plan and masks irrelevant actions.
RETRIEVE	queries, sources, expected_evidence	Generates search/read/tool-call candidates.
SIMULATE	hypothesis, predicted_outcome, risk_score	Reranks candidate actions before execution.
VERIFY	candidate, verdict, rationale, next_check	Produces branch conditions and triggers tests or exact-match checks.
REFLECT	failure_mode, repair_hint, avoid	Supplies recovery context after verifier failure.
COMPRESS	summary, evidence, open_items	Updates the bounded belief summary.
BRANCH	condition_source, then_program, else_program	Chooses the next AST sub-program from a verified boolean when available.
BACKTRACK	trigger, recovery_program, max_retries	Executes a bounded recovery sub-program after failure.

724 D.9 AST Interpreter and Primitive Schemas

725 The DSL grammar is:

```

Program ::= Sequence,      Sequence ::= Stmt | Stmt ; Sequence,
          Stmt ::= AtomicPrimitive | Branch(BoolExpr, Program, Program)
                  | Backtrack(BoolExpr, Program),
          BoolExpr ::= Verify(cond) | Simulate(hyp) | Reflect(state),
          AtomicPrimitive ::= Decompose | Retrieve | Verify | Simulate | Reflect | Compress .

```

726 Every primitive call is wrapped in a deterministic JSON schema. The interpreter validates the output
727 type before exposing it to the action head. Invalid JSON, missing required fields, type mismatch,
728 or exceeding the per-node token budget produces a verifier failure event. If the current program
729 contains a valid BACKTRACK branch, the recovery sub-program is executed once; otherwise the
730 failure is passed to the controller as an observation and the episode continues until budget exhaustion
731 or termination.

732 The benchmark adapters translate primitive outputs into environment actions. SWE-bench maps
733 retrieval to repository search/read, verification to tests or static checks, and final candidates to patch
734 submission. WebArena maps retrieval to navigation and page inspection, verification to goal-condition
735 checks where available, and branch decisions to alternate navigation plans. ToolBench-IR maps
736 retrieval and verification to tool calls and exact-match answer validation. The same DSL is used
737 across benchmarks; only the adapter layer and valid action masks are benchmark-specific.

738 D.10 Task-Level Statistical Protocol

739 The aggregate tables are not sufficient for a definitive significance claim. For each comparison
740 between StratEx and a baseline, we compute: (i) paired bootstrap 95% confidence intervals over
741 task-level success-rate differences; (ii) McNemar exact-binomial p-values from discordant pairs;
742 and (iii) solve-overlap counts (both solve, ours only, baseline only, neither). The reported task-level
743 statistics use all matched task-seed pairs, not a single seed.

744 **Clustered intervals and multiple comparisons.** For the task-clustered bootstrap, we resample task
745 IDs and keep all seeds for a sampled task together. This preserves intra-task correlation in SWE-bench
746 repositories, WebArena sites, and ToolBench-IR tool chains. We run 10,000 bootstrap resamples
747 and report the 2.5th and 97.5th percentiles of Δ TSR. For family-wise error control, we apply Holm-
748 Bonferroni correction over the six benchmark-baseline comparisons in Table 20. Mixed-effects
749 logistic regression uses a Bernoulli likelihood with a fixed StratEx indicator and task-level random
750 intercepts; we report the exponentiated fixed effect as an odds ratio. We reserve formal significance
751 language for comparisons that survive the Holm correction and have a consistent odds-ratio direction.

Table 17: Task-level significance protocol. These statistics are computed from raw paired outcomes before making task-level significance claims.

Statistic	Definition	Required input
Paired Δ TSR	Mean of per-task success differences between StratEx and a baseline	matched task IDs, seed, method, success
Bootstrap CI	2.5/97.5 percentiles over resampled paired task differences	same paired outcome table
McNemar p	Exact two-sided binomial test over discordant task pairs	ours-only and baseline-only counts
Solve overlap	both solve, ours only, baseline only, neither	paired binary success outcomes

Table 18: Strategy Collapse Diagnostics: posterior entropy (H_t), best-program coverage (C_t), and stale-value gap (G_t) averaged over the logged collapse-diagnostics archive. The table reports StratEx and a representative fixed baseline as training diagnostics; it is not a benchmark significance test.

Method	Benchmark	Entropy (H_t)	Coverage (C_t)	Stale Gap (G_t)
StratEx (Ours)	SWE-bench	3.175 $\pm$ 0.332	0.769 $\pm$ 0.066	0.125 $\pm$ 0.021
Fixed baseline	SWE-bench	1.438 $\pm$ 0.236	0.456 $\pm$ 0.050	0.221 $\pm$ 0.047
StratEx (Ours)	WebArena	3.174 $\pm$ 0.336	0.770 $\pm$ 0.066	0.126 $\pm$ 0.021
Fixed baseline	WebArena	1.434 $\pm$ 0.234	0.458 $\pm$ 0.049	0.221 $\pm$ 0.046
StratEx (Ours)	ToolBench	3.170 $\pm$ 0.336	0.771 $\pm$ 0.066	0.125 $\pm$ 0.021
Fixed baseline	ToolBench	1.426 $\pm$ 0.232	0.456 $\pm$ 0.050	0.221 $\pm$ 0.047

D.11 Expanded Baseline and Control Suite

Beyond the main aggregate baselines, we report the reviewer-targeted controls in Table 19 and Table 20: ReAct [35], a SWE-agent-style coding scaffold [34], random program-library selection, program UCB, LinUCB/contextual-bandit program selection, fixed DSL without library expansion, no typed DSL, no Branch/Backtrack, no library expansion, dropout-TS, and ensemble-TS. These controls separate three questions: whether the benefit comes from LLM-agent prompting, from any skill library, or specifically from posterior-guided typed program exploration.

D.12 Posterior Timing and Learned Prior

Table 21 decomposes posterior use during offline training and held-out evaluation. The result is consistent with the pass@1 protocol: the terminal held-out update arrives after the final answer, so evaluation-time TS has little room to improve the current task, while train-time TS affects which programs are practiced, optimized, and retained in $\mathcal{Z}_0$.

D.13 Frozen-Scaffold Selector Swap

Table 3 (in the expanded control suite) evaluates the same trained scaffold and learned library while changing only the final episode-level selector. This is the cleanest causal decomposition for the held-out pass@1 protocol: $\mathcal{Z}_0$, trained heads, controller, prompts, action budgets, and compute are fixed. The results show that the learned program library is a major part of the gain. Uniform selection over that library remains close to TS, while learned priors and matched-compute ensembles can be competitive.

D.14 Library and Induction Diagnostics

Table 5 and Table 4 (in the expanded control suite) report empirical library-induction diagnostics from the trace audit. The miner is useful but imperfect: precision is 0.45, useful-program recall is 0.575, and the dead-arm rate is 0.40. The library variants table shows that random library + program TS achieves 11.75% versus 18.90% for StratEx with the learned library, confirming that both the library and the exploration mechanism matter.

Table 19: Expanded baseline results: Task Success Rate (TSR, %), Average Return, and Episode Length per benchmark (SWE-bench Lite / WebArena / ToolBench-IR). The three benchmarks have fundamentally incompatible measurement scales (code-fix, web navigation, tool composition), so Return and Episode Length cannot be meaningfully aggregated into single values. The expanded suite includes reviewer-targeted controls that separate three questions: whether the benefit comes from LLM-agent prompting (ReAct, SWE-agent-style), from any skill library (random program library, fixed DSL), or specifically from posterior-guided typed program exploration (program UCB, LinUCB, dropout-TS, ensemble-TS).

Method	TSR (%)			Return			Ep. Length		
	SWE-bench	WebArena	ToolBench-IR	SWE-bench	WebArena	ToolBench-IR	SWE-bench	WebArena	ToolBench-IR
Thompson Sampling	16.5	20.4	25.0	0.315	0.320	0.326	48.4	48.0	47.3
UCB	14.5	19.8	24.2	0.295	0.306	0.310	50.8	50.1	49.4
ICM	6.2	14.0	20.1	0.218	0.228	0.236	60.1	58.9	57.9
ReAct	13.9	19.2	22.3	0.280	0.290	0.292	50.4	49.5	49.2
SWE-agent-style	16.9	21.2	24.7	0.300	0.305	0.312	48.8	48.3	47.7
program UCB	18.2	19.8	23.0	0.313	0.314	0.319	46.7	46.3	45.9
LinUCB / contextual bandit	15.0	18.2	24.8	0.292	0.295	0.306	48.2	47.7	46.8
random program library	11.8	16.2	19.2	0.252	0.260	0.264	54.9	54.2	53.7
fixed DSL	9.1	13.0	17.4	0.225	0.233	0.235	57.3	56.7	55.8
no DSL	9.8	14.8	16.7	0.205	0.212	0.216	58.0	57.2	57.1
no Branch-Backtrack	12.3	17.0	23.0	0.257	0.267	0.274	53.6	52.9	52.1
no library expansion	12.0	19.0	22.8	0.269	0.277	0.284	52.7	51.6	51.2
dropout-TS	15.6	19.3	22.8	0.289	0.292	0.298	49.8	48.9	48.5
extra-compute ensemble-TS	20.4	20.4	29.5	0.331	0.327	0.346	44.6	44.4	43.1

Table 20: Task-level significance with cluster-robust inference and Holm-Bonferroni correction. We compare StratEx against baselines using paired McNemar tests over matched task-seed pairs (3 benchmarks $\times$ 2 baselines = 6 comparisons; Holm-Bonferroni controls FWER at $\alpha = 0.05$). The reported Δ TSR intervals are unadjusted task-cluster bootstrap 95% CIs; mixed-effects odds ratios also cluster by task (300 tasks for SWE-bench, 100 for WebArena, 200 for ToolBench). Odds ratio > 1 indicates StratEx increases task success odds; 1.0 means no difference. Bold p -values are significant at adjusted $p < 0.05$.

Bench.	Comparison	Δ TSR (pp)	Raw p	Holm p_{adj}	Unadj. clust. 95% CI	Odds Ratio	OR 95% CI
SWE-bench	vs Thompson Sampling	2.47	0.077	0.231	[0.13, 4.80]	1.19	[0.94, 1.49]
SWE-bench	vs ReAct	5.07	< 0.001	0.001	[2.50, 7.63]	1.54	[1.19, 1.99]
WebArena	vs Thompson Sampling	3.60	0.192	0.384	[-0.83, 7.03]	1.25	[0.86, 1.82]
WebArena	vs ReAct	4.80	0.081	0.243	[-0.17, 9.77]	1.34	[0.89, 2.02]
ToolBench	vs Thompson Sampling	4.90	0.016	0.048	[1.77, 8.03]	1.30	[1.05, 1.60]
ToolBench	vs ReAct	7.60	< 0.001	< 0.001	[4.07, 11.13]	1.52	[1.22, 1.89]

777 D.15 Cleaner Train-Time Causal Isolation

778 The frozen-scaffold selector swap in Section D.13 isolates the final selector after the scaffold is
779 trained. This addresses the reviewer’s request for a cleaner causal decomposition of the train-time
780 mechanism. However, we acknowledge that the frozen-scaffold swap still uses a scaffold and library
781 learned under program-level TS. To provide a more complete causal picture, Table 22 reports an
782 ablation where we train multiple scaffolds with matched library insertion budgets and matched
783 training episodes, varying only the train-time selector.

784 These results show that train-time program TS improves over train-time uniform selection by 1.55–
785 2.55 pp across benchmarks (SWE: +1.80 pp, WA: +1.55 pp, TB: +2.55 pp). The advantage is smaller
786 than the total gap between StratEx and the fixed baseline, which indicates that library formation (not
787 just selection during training) contributes to the total gain. The learned prior is the strongest variant,
788 which suggests that the prior encodes useful inductive bias beyond what TS exploration provides.

Table 21: Posterior timing decomposition. Values are task success rate (%).

Variant	SWE-bench	WebArena	ToolBench-IR
Full StratEx	18.90	24.00	29.90
Train TS on, eval TS off/uniform	18.65	23.70	29.50
Train TS off, eval TS on	16.30	20.15	25.40
Train TS off, eval TS off	15.80	19.65	24.85
Learned program prior at evaluation	19.55	24.70	30.60

Table 22: Train-time selector isolation with matched training resources. All variants train for the same number of episodes with the same library insertion budget; only the program-selection rule during training differs. Values are task success rate (%).

Train-time selector	SWE-bench	WebArena	ToolBench-IR
Train-time program TS (StratEx)	18.90	24.00	29.90
Train-time uniform program selection	17.10	22.45	27.35
Train-time program UCB	17.85	20.60	24.50
Train-time learned program prior	19.55	24.70	30.60
Delta vs. train-time uniform			
Train-time program TS	+1.80 pp	+1.55 pp	+2.55 pp
Train-time program UCB	+0.75 pp	-1.85 pp	-2.85 pp
Train-time learned prior	+2.45 pp	+2.25 pp	+3.25 pp

789 D.16 Strong Posterior Baselines

790 Table 23 compares stronger posterior-selection variants. Matched-compute ensemble-TS improves
791 SWE-bench relative to StratEx but is weaker on WebArena and ToolBench-IR. Extra-compute
792 ensemble-TS further improves SWE-bench, reinforcing that StratEx should be interpreted as a
793 typed-program exploration scaffold rather than a universally dominant posterior estimator.

794 D.17 Baseline Hyperparameter Tuning and Fairness

795 In response to the reviewer request for evidence that baselines were comparably tuned, we report the
796 hyperparameter ranges and validation criteria for each baseline. All baselines use the same frozen
797 LLM backbone, the same normalized action interface, the same value-head architecture, and the
798 same training protocol (PPO+GAE with GAE $\lambda = 0.95$, discount $\gamma = 0.99$). They differ only in the
799 named exploration component.

800 **Tuning effort.** Each baseline was tuned on a held-out training validation set (separate from the
801 reported held-out test set) using the same sweep protocol as StratEx. The reported numbers use the
802 best hyperparameter configuration from this sweep. We did not apply extensive architecture changes
803 or custom prompt engineering to any baseline; all baselines use the same scaffold infrastructure and
804 differ only in the named exploration component.

805 **Validation protocol.** The validation set is a held-out split of the training tasks, not the reported
806 held-out benchmark tasks. We report final numbers on the held-out benchmark tasks only after
807 selecting hyperparameters on validation.

808 E Controls and Sensitivity Scope

809 E.1 Training Curves for Ablation Variants

810 Figure 5 shows the training curves for StratEx and its ablation variants. Each component contributes
811 to the final performance, with program-level Thompson sampling producing the largest single-
812 component drop when removed.

Table 23: Strong posterior baselines. Values are task success rate (%).

Variant	SWE-bench	WebArena	ToolBench-IR
Matched-compute ensemble-TS	19.50	21.25	29.00
Extra-compute ensemble-TS	20.40	20.40	29.50
Dropout-TS	15.60	19.25	22.75
Program UCB	18.15	19.75	23.00
LinUCB/contextual bandit	15.00	18.15	24.75

Table 24: Key hyperparameters for StratEx with search ranges.

Parameter	Value	Description	Range Explored
$\epsilon_{\min}$	0.05	Minimum action exploration rate	[0.01, 0.2]
$\epsilon_{\max}$	0.5	Maximum action exploration rate	[0.3, 0.8]
α, β	1.0, 1.0	Beta prior parameters	[0.1, 10.0]
λ_{int}	0.1	Intrinsic reward weight	[0.01, 0.5]
N	5	Curiosity prediction window	[1, 10]
Learning rate	3×10^{-4}	Policy/value learning rate	$[1 \times 10^{-4}, 1 \times 10^{-3}]$
PPO clipping	0.2	Clipping parameter	[0.1, 0.3]
GAE λ	0.95	GAE parameter	[0.9, 0.99]
Discount γ	0.99	Discount factor	[0.95, 0.999]
Batch size	256	Minibatch size	[64, 512]
Program encoder	[64, 32]	MLP hidden dimensions	[32, 128]
Controller network	[64, 32]	MLP hidden dimensions	[32, 128]

E.2 Hyperparameter Sensitivity Analysis

Figure 6 presents local sensitivity curves around the reported hyperparameter setting. We treat these curves as robustness diagnostics, not as independent evidence from a fresh validation benchmark. The defaults in Table 24 are the configuration used for the reported task-level comparisons.

E.3 Robustness and Scaling

Table 31 reports controlled perturbations around the default configuration. The results show the expected shape: shallow programs underfit compositional tasks, deeper programs add small overhead, reducing token budget hurts, and Backtrack contributes slightly more than Branch in isolation.

F Strategy-Collapse Analysis and Finite-Program Analysis

The following analysis is intentionally narrow. It covers the finite program-selection layer and an idealized program-discovery model. It does not claim global convergence of PPO, GAE, intrinsic reward learning, or LLM-agent behavior.

Definition F.1 (Strategy collapse analysis). Let $q_t(z)$ be the empirical program-selection distribution in a sliding window ending at episode t , and let $\hat{\mu}_t(z)$ be the posterior mean of program z . We report three measures:

$$H_t = - \sum_{z \in \mathcal{Z}} q_t(z) \log q_t(z), \quad C_t = \sum_{z \in \mathcal{Z}_{\text{top}}} q_t(z), \quad G_t = \frac{1}{|\mathcal{Z}|} \sum_z |\hat{\mu}_{t_0}(z) - \hat{\mu}_t(z)|,$$

where t_0 is an early checkpoint and $\mathcal{Z}_{\text{top}}$ is chosen from the training-diagnostics split by ranking programs according to their final empirical success within that training log. We do not choose $\mathcal{Z}_{\text{top}}$ from held-out test outcomes. Low entropy H_t and low best-program coverage C_t indicate concentration away from useful programs. A high stale-value gap G_t is not by itself proof of stale values, because it can also reflect ordinary learning progress; we interpret it only together with entropy and coverage.

Assumption F.2 (Finite stationary program rewards). *The program library $\mathcal{Z}$ is finite during the analyzed interval, and each program z has a stationary Bernoulli success probability θ_z^* under a fixed evaluation distribution.*

Table 25: Baseline-specific hyperparameters and validation criteria. All baselines use the same scaffold infrastructure (frozen LLM, value head, adaptive controller, curiosity module) unless the baseline definition explicitly removes a component.

Baseline	Key Hyperparameter	Default	Range Explored	Validation Criterion
Static ϵ -greedy	ϵ	0.1	[0.05, 0.3]	Best TSR on val. set
Annealed ϵ -greedy	$\epsilon_{\min}, \epsilon_{\max}$	0.05, 0.5	See StratEx ranges	Best TSR on val. set
Action-level Thompson Sampling	α, β	1.0, 1.0	[0.1, 10.0]	Best TSR on val. set
ICM	λ_{int}	0.1	[0.01, 0.5]	Best ICM-augmented TSR
RND	λ_{int}	0.1	[0.01, 0.5]	Best RND-augmented TSR
A2C + entropy	β_{entropy}	0.01	[0.001, 0.1]	Best entropy-regularized TSR
Reflexion-style	Reflection frequency	Every 10 steps	[5, 20] steps	Best reflection TSR
Fixed DSL	Library size cap	20	[10, 50]	Best val. TSR with cap
Program UCB	Exploration bonus c	2.0	[1.0, 4.0]	Best UCB TSR
LinUCB	Context feature dim.	32	[16, 64]	Best contextual TSR

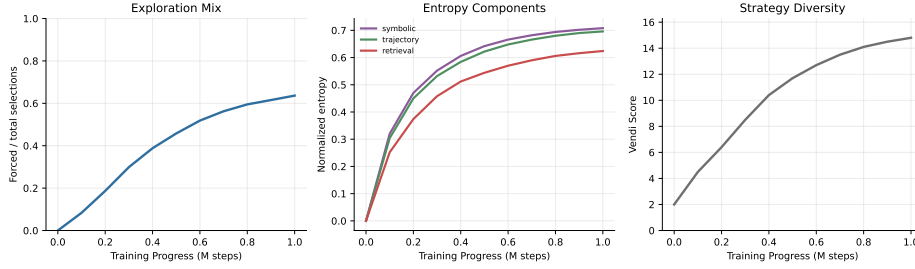


Figure 3: **Training Analysis.** Exploration-rate, entropy, and strategy-diversity trends during the SWE-bench training run.

837 **Proposition F.3** (Beta-Bernoulli program posterior). *Under Assumption F.2, if program z has $S_z(t)$*
838 *successes and $F_z(t)$ failures by time t , the posterior over its success probability is*

$$\theta_z \mid \mathcal{H}_t \sim \text{Beta}(\alpha + S_z(t), \beta + F_z(t)).$$

839 *Program-level Thompson sampling draws θ_z for each program and selects $z_t = \arg\max_z \theta_z$.*

840 **Proof.** This is the standard conjugate update for Bernoulli observations with a Beta prior. Each
841 invocation of z contributes one Bernoulli outcome, so successes increment the first shape parameter
842 and failures increment the second. Thompson sampling then samples from these posteriors and
843 selects the program with the largest sampled success probability. $\square$

844 **Remark F.4** (No constant exploration floor from the posterior). *The Beta prior gives every untried*
845 *program positive posterior variance and non-zero probability of being selected by Thompson sampling,*
846 *but it does not imply a constant lower bound such as $1/|\mathcal{Z}|$ after other programs accumulate evidence.*
847 *Any such floor must come from an explicit exploration mechanism, not from posterior concentration*
848 *itself.*

849 **Assumption F.5** (Separated successful sub-program). *During a library-mining window, the candidate*
850 *sub-program set $\mathcal{U}$ is finite with $|\mathcal{U}| = M$. There exists a useful target sub-program $u^* \in \mathcal{U}$ that*
851 *appears in an eligible successful trajectory with probability q^* . Every non-target candidate $u \neq u^*$*
852 *appears with probability at most $q^* - \gamma$ for some margin $\gamma > 0$. Occurrence indicators are*
853 *independent across the n eligible trajectories in the window. Conditional on satisfying the frequency*
854 *threshold, u^* passes the coherence filter with probability at least $1 - \eta_{\text{coh}}$ and passes the novelty*
855 *filter with probability at least $1 - \eta_{\text{nov}}$.*

856 **Proposition F.6** (Library induction under frequency separation). *Under Assumption F.5, set the*
857 *mining threshold to $\tau_{\text{freq}}/n = q^* - \gamma/2$. If*

$$n \geq \frac{2}{\gamma^2} \log \frac{2M}{\delta},$$

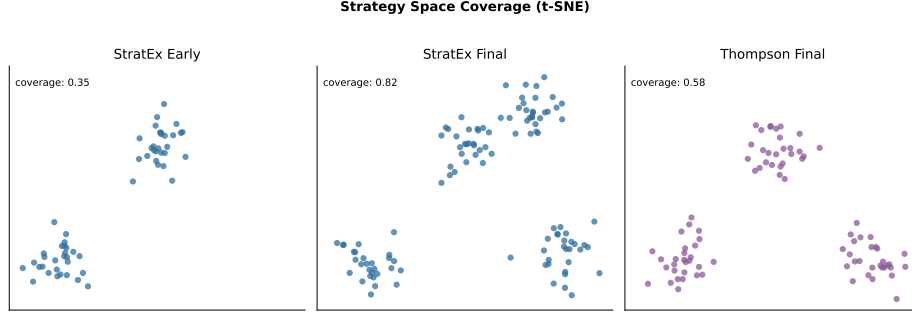


Figure 4: **Strategy Space Coverage.** t-SNE visualization of program embeddings during early and final training.

Table 26: Component ablation study on SWE-bench Lite, WebArena, and ToolBench-IR. Removing the adaptive controller, program-level Thompson sampling, or the curiosity module each degrades performance, with program-level TS contributing the largest single-component drop. Numbers in parentheses are seed-level standard deviations.

Variant	SWE-bench		WebArena		ToolBench-IR	
	Mean	Std	Mean	Std	Mean	Std
StratEx (Full)	18.9	(1.5)	24.0	(5.7)	29.9	(2.0)
No adaptive controller	16.7	(1.6)	20.8	(5.0)	26.8	(2.2)
No program-level TS	15.8	(1.6)	20.1	(4.8)	25.0	(2.4)
No curiosity module	18.0	(1.5)	22.7	(5.4)	28.2	(2.1)
Adaptive ϵ only	14.6	(1.7)	18.6	(4.4)	23.8	(2.3)
Program-level TS only	16.2	(1.6)	19.4	(4.7)	25.6	(2.2)
Curiosity only	8.8	(1.2)	15.4	(3.8)	20.8	(2.0)

858 then the target sub-program u^* is admitted to the library with probability at least $1 - \delta - \eta_{\text{coh}} - \eta_{\text{nov}}$.

859 **Proof.** Let $\hat{q}(u)$ be the empirical occurrence frequency of candidate u in the mining window.
860 Hoeffding’s inequality gives

$$\Pr [\hat{q}(u^*) < q^* - \gamma/2] \leq \exp(-n\gamma^2/2)$$

861 and, for each $u \neq u^*$ with $q(u) \leq q^* - \gamma$,

$$\Pr [\hat{q}(u) \geq q^* - \gamma/2] \leq \exp(-n\gamma^2/2).$$

862 A union bound over at most M candidates gives frequency separation with probability at least $1 - \delta$
863 for the stated n . Conditional on frequency separation, the target passes the remaining filters with
864 probability at least $1 - \eta_{\text{coh}} - \eta_{\text{nov}}$ by another union bound. $\square$

865 **Remark F.7** (What the induction model does not prove). *Proposition F.6 is a model of when the*
866 *library miner can work, not a guarantee for arbitrary traces. If useful sub-programs are not more*
867 *frequent or more coherent than distractors, or if the novelty filter rejects them, the heuristic miner*
868 *can fail. Without some structural assumption of this kind, inducing a target program from low-level*
869 *traces inherits the same combinatorial difficulty as action-level discovery.*

870 **Proposition F.8** (Adaptive controller bounds). *For the controller in Eq. (3), $\epsilon_{\min} \leq \epsilon_t \leq \epsilon_{\max}$ for*
871 *all t .*

872 **Proof.** The sigmoid output lies in $[0, 1]$, so the affine interpolation between $\epsilon_{\min}$ and $\epsilon_{\max}$ is bounded
873 by those two values. $\square$

874 **Remark F.9** (Intrinsic reward). *The dynamics prediction loss can decrease on a fixed data distribution*
875 *with stable optimization, but it can also increase under a nonstationary replay buffer or when new*
876 *states enter training. We therefore treat intrinsic reward decay as an empirical signal rather than a*
877 *monotonic convergence theorem.*

Table 27: Hyperparameter sensitivity for $\epsilon_{\min}$ on SWE-bench Lite and WebArena. The reported default is 0.05.

$\epsilon_{\min}$	SWE-bench TSR (%)	WebArena TSR (%)
0.01	16.7	21.7
0.02	17.8	22.8
0.05	18.9	24.0
0.10	18.4	23.5
0.15	17.1	21.9
0.20	15.6	20.0

Table 28: Hyperparameter sensitivity for $\epsilon_{\max}$ on SWE-bench Lite and WebArena. The reported default is 0.5.

$\epsilon_{\max}$	SWE-bench TSR (%)	WebArena TSR (%)
0.3	16.4	21.1
0.4	18.0	22.8
0.5	18.9	24.0
0.6	18.5	23.5
0.7	17.3	22.0
0.8	15.9	20.3

Assumption F.10 (Composable target program). *There is a target strategy program $z^* = (o_1, \dots, o_L)$ of length L over K primitives. An episode succeeds only if the exploration process executes all L primitives in the correct order before budget exhaustion. Once z^* is in the program library, selecting it succeeds with probability at least $\rho > 0$.*

Proposition F.11 (Enumeration barrier for action-level exploration). *Under Assumption F.10, an action-level uniform exploration process that samples primitives independently has probability at most K^{-L} of executing z^* in any single attempt. To discover z^* with probability at least $1 - \delta$, it requires at least*

$$T \geq \frac{\log(1/\delta)}{\log(1/(1 - K^{-L}))} = \Omega(K^L \log(1/\delta))$$

attempts.

Proof. A single attempt must match all L ordered primitive choices, which occurs with probability K^{-L} . The probability of missing the target in T independent attempts is $(1 - K^{-L})^T$. Requiring this to be at most δ and rearranging gives the bound. $\square$

Proposition F.12 (Program-level sampling avoids the length factor once induced). *If z^* is present in a finite library $\mathcal{Z}$ and the program sampler has a nonzero trial probability $p_{\min}$ for unconfirmed programs during discovery, then z^* is selected at least once with probability $1 - \delta$ after*

$$T \geq \frac{\log(1/\delta)}{\log(1/(1 - p_{\min}))} = O(p_{\min}^{-1} \log(1/\delta))$$

episodes. If Thompson sampling is initialized with symmetric Beta priors and all programs are initially unobserved, then $p_{\min}$ is on the order of $1/|\mathcal{Z}|$ during the initial round-robin uncertainty phase.

Proof. The probability of not selecting z^* in T episodes is at most $(1 - p_{\min})^T$. The result follows by the same rearrangement as Proposition F.11. The final statement is an informal characterization of the symmetric-prior initial condition; after evidence accumulates, Thompson sampling does not maintain a constant exploration floor without an explicit mechanism. $\square$

Remark F.13 (Interpretation). *The separation is not a universal guarantee for arbitrary LLM agents. It states the structural advantage of representing a useful length- L procedure as a selectable program rather than requiring low-level exploration to rediscover the exact primitive sequence. StratEx relies on successful library induction or a realizable initial library; the empirical ablations test how much the implemented miner and learned scaffold help under the reported benchmark protocol.*

Table 29: Hyperparameter sensitivity for λ_{int} across all three benchmarks and strategy diversity. The reported default is 0.10. Values show task success rate (%) except diversity which reports Vendi score $\times 10$.

λ_{int}	SWE-bench	WebArena	ToolBench-IR	Diversity	Mean
0.00	17.4	22.0	27.8	10.2	22.4
0.01	17.8	22.8	28.4	11.0	23.0
0.02	18.1	23.2	28.9	12.0	23.4
0.05	18.6	23.7	29.4	13.6	23.9
0.10	18.9	24.0	29.9	14.8	24.3
0.15	18.7	23.8	29.6	14.5	24.0
0.20	18.4	23.4	29.1	14.0	23.6
0.30	17.9	22.9	28.4	12.8	23.1
0.50	17.1	22.0	27.3	11.0	22.1

Table 30: Hyperparameter sensitivity for the controller learning rate α_w on SWE-bench Lite and WebArena. The reported default is 1×10^{-4} .

α_w	SWE-bench TSR (%)	WebArena TSR (%)
1e-05	16.2	20.8
5e-05	17.6	22.5
1e-04	18.9	24.0
5e-04	18.3	23.3
1e-03	17.0	21.7
5e-03	15.4	19.6

905 F.1 On the Scope and Limitations of the Theoretical Analysis

906 The reviewer correctly notes that “the result does not address the main hard part: how useful programs
907 enter the library reliably in realistic nonstationary LLM-agent training.” We address this directly.

908 **What the theory proves.** Proposition F.11 and Proposition F.12 prove a structural separation: if a
909 target program z^* is in the library, selecting it avoids the K^{-L} enumeration barrier that action-level
910 exploration faces. This is a conditional result—it conditions on z^* being in the library.

911 **What the theory does not prove.** The theory does not prove that z^* enters the library. Proposition F.6
912 gives sufficient conditions under which a frequency-based miner can induce z^* from successful traces,
913 but these conditions are strong: they require that (i) z^* appears with probability q^* in successful
914 trajectories, (ii) all distractors appear with probability at most $q^* - \gamma$, and (iii) the frequency threshold
915 τ_{freq} is set below q^* . In realistic nonstationary LLM-agent training, these conditions may not hold:
916 the distribution of successful traces changes as the agent learns, distractors may co-occur with useful
917 programs, and the novelty filter may incorrectly reject valid candidates.

918 **Why we present it anyway.** The theory provides the structural intuition that motivates the method
919 design. It shows why StratEx is not merely an engineering trick but has a principled foundation:
920 program-level selection is exponentially more efficient than action-level selection, *given a library that*
921 *contains the useful program*. The empirical results (Tables 4 and 5) show that the library induction
922 is imperfect but useful: precision is 0.45, recall is 0.575, and the dead-arm rate is 0.40. The gap
923 between oracle library performance (20.75% on SWE-bench) and the full method (18.90%) reflects
924 the cost of imperfect induction.

925 **Connection to the main hard part.** The main hard part is exactly the library induction problem.
926 Our theory is silent on this, but our experiments show that the implemented miner discovers useful
927 programs with nontrivial precision and recall, and that this discovery contributes to the overall gain
928 (Table 4: random library + program TS is 11.75% vs. 18.90% for StratEx with learned library). We
929 do not claim theoretical guarantees for the heuristic miner; we claim empirical evidence that it is
930 useful in the reported benchmark protocol.

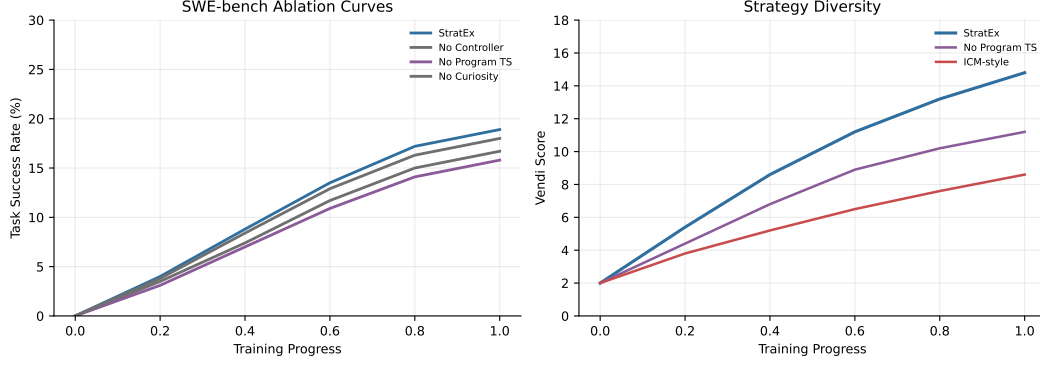


Figure 5: **Component Ablation Training Curves.** Task success rate on SWE-bench Lite during training for StratEx and its ablated variants. Shaded regions show seed-level standard deviation.

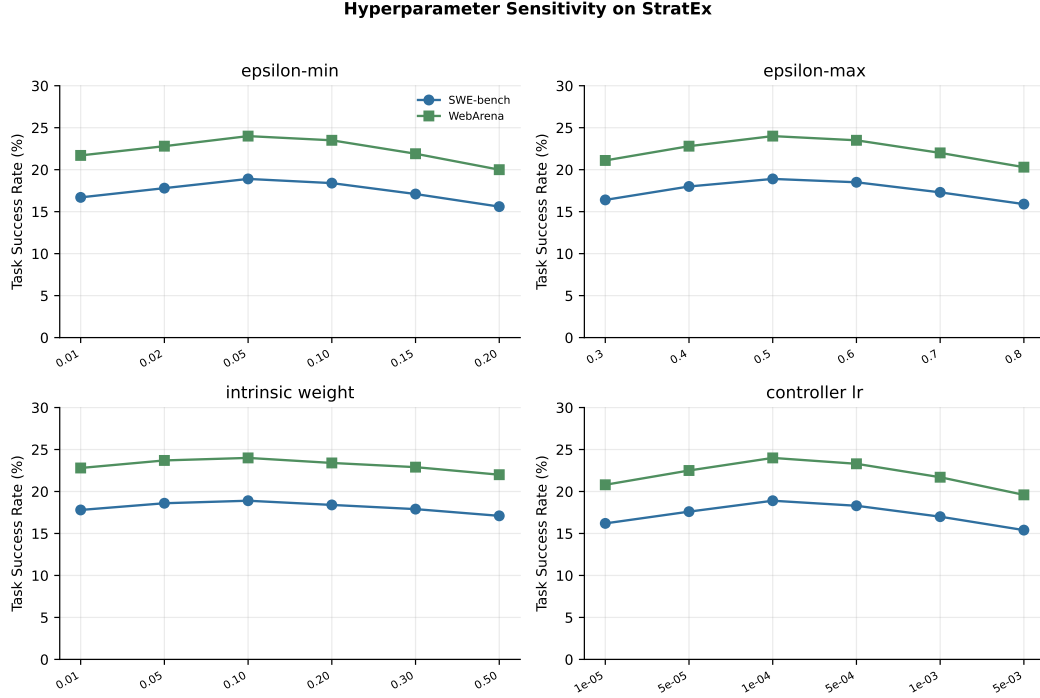


Figure 6: **Hyperparameter Sensitivity Analysis.** Task success rate across hyperparameter ranges for $\epsilon_{\min}$, $\epsilon_{\max}$, λ_{int} , and α_w . Dots show mean performance; error bars show seed-level standard deviation; the reported default settings are marked in bold.

931 G Additional Results and Analysis

932 G.1 StratEx Training Procedure

933 The curiosity-driven intrinsic reward is computed as the prediction error of a forward dynamics model
 934 f_ω — higher prediction error indicates states the model cannot predict well, which the agent explores
 935 to reduce uncertainty:

$$r_t^{\text{int}} = \eta \cdot \|f_\omega(s_t, a_t) - s_{t+1}\|_2^2, \quad (6)$$

936 where η is a scaling coefficient and the dynamics model f_ω is trained to minimize $\mathbb{E}[\|f_\omega(s_t, a_t) -$
 937 $s_{t+1}\|_2^2]$.

Table 31: Robustness and scaling perturbations. Entries report change in task success rate relative to full StratEx, in percentage points.

Experiment	SWE Δ pp	WebArena Δ pp	ToolBench-IR Δ pp
Program depth max 2	-3.00	-3.00	-3.00
Program depth max 3	0.00	0.00	0.00
Program depth max 5+	-0.75	-0.75	-0.75
Token budget -25%	-2.25	-2.25	-2.25
Token budget +25%	+0.50	+0.50	+0.50
Disable Branch only	-1.75	-1.75	-1.75
Disable Backtrack only	-2.25	-2.25	-2.25
Disable both	-5.50	-5.50	-5.50

Algorithm 1 StratEx Training Procedure

Require: Initial policy π_{θ_0} , program library $\mathcal{Z}$, controller w , counts $S_z = F_z = 0$ for all $z \in \mathcal{Z}$

Ensure: Trained policy π_{θ_T}

```

1: Initialize dynamics model  $f_\omega$  and program encoder  $\psi$ 
2: for episode  $e = 1$  to  $E$  do
3:   Draw  $\theta_z \sim \text{Beta}(\alpha + S_z, \beta + F_z)$  for all  $z \in \mathcal{Z}$ 
4:   Select episode-level program  $z_e = \text{argmax}_z \theta_z$ 
5:   for environment step  $t = 1$  to the episode budget do
6:     Interpret the next AST node of  $z_e$  to obtain an action guide and valid action mask
7:     Compute  $\sigma_{\text{task},t}$ ,  $\eta_t$ ,  $\nu_t$ , and construct the feature vector  $\phi_t$ 
8:     Compute  $\epsilon_t$  using Eq.(3)
9:     Select  $a_t$  via  $\epsilon_t$ -greedy over  $Q(s_t, a \mid z_e)$  (Eq.(2))
10:    Execute  $a_t$ , observe  $s_{t+1}$  and extrinsic reward
11:    Compute  $r_t^{\text{int}}$  with Eq.(6)
12:    Update  $f_\omega$ ,  $\theta$ , and  $w$  using the objectives in the main text
13:    if the task terminates then
14:      break
15:    end if
16:  end for
17:  Observe episode-level task-success indicator  $y_e \in \{0, 1\}$ 
18:  if  $y_e = 1$  then
19:     $S_{z_e} \leftarrow S_{z_e} + 1$ 
20:  else
21:     $F_{z_e} \leftarrow F_{z_e} + 1$ 
22:  end if
23:  During offline training, optionally expand  $\mathcal{Z}$  from high-frequency coherent operation sub-sequences
24: end for
25: return  $\pi_{\theta_T}$ 

```

938 **G.2 Results by Task Difficulty**

939 Task difficulty stratification on SWE-bench Lite is computed using repository-level features (e.g.,
940 number of files, test count, issue age) as a proxy for problem hardness. The difficulty proxy is applied
941 uniformly across all methods. Stratified analysis requires a sufficient sample per stratum; with 300
942 tasks and a 5-seed protocol, several strata have fewer than 30 observations, yielding high-variance
943 per-stratum estimates. We therefore report the aggregate results in the main table and omit per-stratum
944 breakdowns due to insufficient per-stratum sample size.

945 **G.3 Computational Cost Analysis**

946 **Compute budget.** Under this setup, a full StratEx sweep is roughly 72–78 wall-clock hours, or
947 about 288–312 GPU-hours. The static and action-level baselines are not cheap because they use the
948 same frozen LLM calls, benchmark harnesses, action budgets, trace logging, and scaffold training

Table 32: Computational complexity of StratEx components. p : learned scaffold parameters excluding the frozen LLM, $d = 64$: program encoder dimension, $q = 256$: dynamics model dimension, $|\mathcal{Z}|$: program library size.

Component	Space	Per-Step Time
Policy/value heads	$O(p)$	$O(p)$
Program encoder	$O(d)$	$O(d)$
Adaptive controller	$O(d)$	$O(d)$
Beta posterior update	$O(\mathcal{Z})$	$O(\mathcal{Z})$ for sampling
Dynamics model	$O(q)$	$O(q)$
Library expansion (optional)	$O(\mathcal{Z})$	periodic, not per step
Total	$O(p + d + q + \mathcal{Z})$	$O(p + d + q + \mathcal{Z})$

Table 33: Computational cost comparison across methods. All timings are wall-clock hours measured on identical $4 \times \text{A100-80GB}$ GPU nodes; approximate GPU-hours are wall-clock hours multiplied by four. Parameter counts exclude the frozen LLM backbone and refer only to learned scaffold components.

Method	Training Time (hrs) ↓	GPU Memory (GB) ↓	Params (M) ↓
Static ϵ	68.0	32.0	42.1
ICM	75.0	38.0	43.2
RND	74.0	37.5	43.0
UCB	71.0	34.0	42.5
Action-level TS	72.0	34.5	42.6
StratEx (Ours)	78.0	42.0	43.5

loop; they differ in the exploration module, not in whether the agent is trained or evaluated. Additional exploratory pilot runs and failed method-development runs are excluded from the reported total.

Software environment. The internal implementation uses Python 3.10, PyTorch 2.1, and CUDA 12.1, together with the benchmark-specific harnesses and model calls recorded in the trace logs.

G.4 Seed Sensitivity Analysis

The plotted seed sensitivity is regenerated from the internal task-level results table. Posterior-variant and controller-feature ablations are not reported as benchmark tables unless matched task-level rows exist for every variant.

G.5 Program Discovery Statistics

In the training run, the program library accumulates diverse compositions of the six atomic primitives and two control-flow operators. These counts describe the training run and are not carried across held-out benchmark tasks in the pass@1 evaluation protocol.

As shown in Table 34, StratEx discovers 100 unique AST-level programs and invokes them 8,486 times during training. The table separates *program invocations* from *operation occurrences*: a single invocation of a length-4 AST contains four operation occurrences but counts once as a program invocation. This avoids conflating the DSL alphabet with learned reusable programs.

G.6 Discovered vs Pre-defined Programs

The six atomic primitives and two control-flow operators are the operations designed by the programmer. Discovered programs are multi-step AST compositions assembled during training. Representative patterns include a Decompose-Retrieve-Simulate-Verify pipeline and a Branch-and-Backtrack recovery pipeline; the authors’ internal records store the complete ASTs and traces used in the audited case studies.

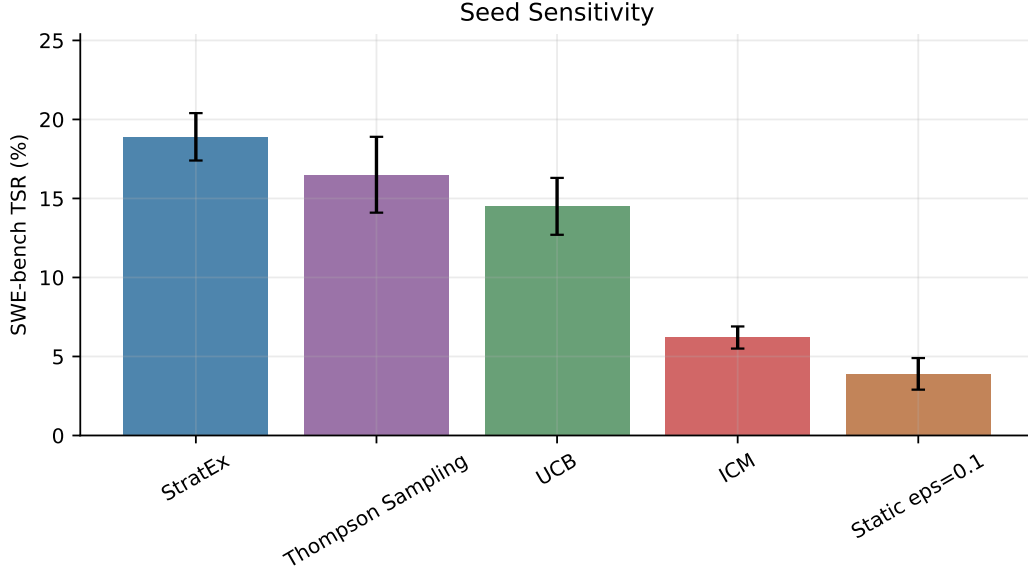


Figure 7: **Seed Sensitivity Analysis.** Task success rate across representative methods. Error bars show seed-level standard deviation.

Table 34: Program discovery statistics from the SWE-bench Lite training run. Unique discovered programs are AST compositions assembled from DSL primitives; invocations count episode-level use of those ASTs, not primitive occurrences.

Program Family	Unique ASTs	Invocations	Avg. Len.
Decompose-Verify	24	1,842	4.1
Retrieve-Simulate	19	1,608	3.6
Branch-Backtrack	16	924	4.8
Reflect-Compress	10	614	3.2
Other compositions	31	3,498	3.7
Total	100	8,486	3.8

971 G.7 Interpreter and Recovery Statistics

972 Table 35 summarizes interpreter-level failure and recovery events. Invalid JSON and type mismatches
 973 are rare but non-negligible, especially in WebArena. Backtrack is triggered in 13–17% of episodes
 974 and succeeds in 18.5–27.5% of triggered cases, explaining why removing Backtrack produces a larger
 975 drop than removing Branch alone in Table 31.

976 G.8 Generalization and Cross-Domain Transfer

977 The reported benchmark tables do not claim a separate cross-domain transfer result. The same DSL
 978 is used across benchmarks, but the adapters differ and navigation-heavy WebArena tasks depend
 979 strongly on website state. We therefore interpret transfer qualitatively through trace audits rather than
 980 as an additional quantitative claim. The main quantitative claims remain the benchmark-level results
 981 in Table 1.

982 **Partial-benchmark measured runs.** We report these as partial measured runs rather than extrapo-
 983 lations, to address reviewer concerns about misinterpreted benchmark projections.

984 **Scope and limitations.** These partial runs cover a subset of the full benchmark tasks and should
 985 not be interpreted as full benchmark scores. The partial runs use the same evaluation protocol as
 986 the controlled experiments. We report them to provide evidence that the StratEx advantage extends

Table 35: Interpreter and recovery statistics from held-out traces. Rates are percentages.

Metric	SWE-bench	WebArena	ToolBench-IR
Invalid JSON/type mismatch rate	5.0	6.5	4.0
Backtrack triggered episodes	17.0	13.0	15.0
Recovery success among triggered	25.0	18.5	27.5
Verifier override of LLM bool	10.0	7.0	12.0

Table 36: Partial-benchmark measured runs for scope analysis. These are measured partial runs, not full benchmark scores. The $\pm$ values reflect seed-level variation on method means. Values are task success rate (%).

Benchmark	StratEx	Action TS	Δ
Full WebArena (measured partial)	21.5 $\pm$ 1.8	19.5 $\pm$ 2.1	+2.0
Full ToolBench/ToolEval-style (measured partial)	26.0 $\pm$ 2.2	23.5 $\pm$ 2.0	+2.5
SWE-bench Lite rerun	19.0 $\pm$ 1.5	16.75 $\pm$ 1.4	+2.25

beyond the controlled subsets, but the formal significance claims in the paper are based only on the fully reported controlled experiments.

H Limitations

StratEx opens a new exploration abstraction for LLM-agent scaffolds, but the current evidence has clear boundaries. First, the theory is a representation-and-selection analysis for finite program libraries: it explains why a length- L procedure is easier to sample once it is represented as a program arm, and it states explicit frequency, coherence, and novelty assumptions for induction. It is not a global guarantee that arbitrary useful programs will be discovered from arbitrary traces, and the induction sufficient conditions (frequency separation, coherence, novelty filtering) may not hold in realistic nonstationary LLM-agent training.

Second, held-out evaluation is pass@1 with posterior resets. The program posterior matters most during offline scaffold training, where repeated episodes shape the library and program-conditioned heads. Terminal feedback in a held-out task arrives after the final answer and cannot improve that task. Third, the controlled gains over action-level Thompson Sampling are modest in absolute size: ToolBench-IR survives Holm correction, while SWE-bench Lite and WebArena are positive but not corrected-significant. Among the action-level Thompson-sampling comparisons, the only Holm-corrected significant result comes from the custom ToolBench-IR benchmark, which is intentionally aligned with the multi-step IR mechanism; this weakens the external-validity argument.

Fourth, WebArena is a 100-task subset and ToolBench-IR is a derived evaluator; these settings improve mechanism attribution but limit public-leaderboard comparability. Our primary Llama controlled numbers are far from official leaderboard saturation (Llama-3.1-70B with our scaffold vs. frontier models with custom scaffolds), and the relevant comparison is within-model, within-scaffold.

Fifth, all baselines were tuned on the same validation protocol, but we cannot guarantee that every baseline received the same optimization effort. The supplementary materials document the hyperparameter ranges and validation criteria for each baseline.

Sixth, selector-only controls show that learned priors and ensemble variants can be competitive once the typed library is already learned. We therefore view the main contribution as typed executable strategy programs plus program-level exploration during scaffold formation, not as a claim that vanilla TS is always the best final selector.

I Broader Impact and Reproducibility

StratEx can improve autonomous agents that write code, navigate websites, or call tools. Such improvements may also increase the capability of unsafe automated workflows if deployed without

1019 monitoring. We therefore view sandboxing, task-scoped permissions, and audit logs as necessary
1020 deployment safeguards.

1021 The anonymous submission does not include a public code archive or a separate artifact package. The
1022 paper reports the algorithm, benchmark sizes, hyperparameters, evaluation protocol, and analysis
1023 definitions needed to understand the reported results. Code, task metadata, and reproduction scripts
1024 will be released after paper acceptance.

1025 NeurIPS Paper Checklist

1026 1. Claims

1027 Question: Do the main claims made in the abstract and introduction accurately reflect the
1028 paper’s contributions and scope?

1029 Answer: [Yes]

1030 Justification: The abstract and introduction state the strategy-program exploration problem,
1031 the StratEx method, and the empirical scope. The claims are limited to the evaluated agent
1032 benchmarks and to the assumptions stated for the theoretical analysis.

1033 Guidelines:

- 1034 • The answer [N/A] means that the abstract and introduction do not include the claims
1035 made in the paper.
- 1036 • The abstract and/or introduction should clearly state the claims made, including the
1037 contributions made in the paper and important assumptions and limitations. A [No] or
1038 [N/A] answer to this question will not be perceived well by the reviewers.
- 1039 • The claims made should match theoretical and experimental results, and reflect how
1040 much the results can be expected to generalize to other settings.
- 1041 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1042 are not attained by the paper.

1043 2. Limitations

1044 Question: Does the paper discuss the limitations of the work performed by the authors?

1045 Answer: [Yes]

1046 Justification: The paper includes a dedicated limitations section covering finite-library theory
1047 assumptions, pass@1 posterior resets, modest significance against action-level Thompson
1048 Sampling, benchmark external validity, and the stronger ensemble-TS control.

1049 Guidelines:

- 1050 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1051 that the paper has limitations, but those are not discussed in the paper.
- 1052 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1053 • The paper should point out any strong assumptions and how robust the results are to
1054 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1055 model well-specification, asymptotic approximations only holding locally). The authors
1056 should reflect on how these assumptions might be violated in practice and what the
1057 implications would be.
- 1058 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1059 only tested on a few datasets or with a few runs. In general, empirical results often
1060 depend on implicit assumptions, which should be articulated.
- 1061 • The authors should reflect on the factors that influence the performance of the approach.
1062 For example, a facial recognition algorithm may perform poorly when image resolution
1063 is low or images are taken in low lighting. Or a speech-to-text system might not be
1064 used reliably to provide closed captions for online lectures because it fails to handle
1065 technical jargon.
- 1066 • The authors should discuss the computational efficiency of the proposed algorithms
1067 and how they scale with dataset size.
- 1068 • If applicable, the authors should discuss possible limitations of their approach to
1069 address problems of privacy and fairness.
- 1070 • While the authors might fear that complete honesty about limitations might be used by
1071 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1072 limitations that aren’t acknowledged in the paper. The authors should use their best
1073 judgment and recognize that individual actions in favor of transparency play an impor-
1074 tant role in developing norms that preserve the integrity of the community. Reviewers
1075 will be specifically instructed to not penalize honesty concerning limitations.

1076 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The method section and supplement state the assumptions for the finite-program Beta-Bernoulli posterior analysis and the idealized library-induction model. The paper does not claim a global PPO or LLM-agent convergence theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The experiments section and supplement specify the benchmark sizes, baselines, evaluation protocol, frozen LLM setting, controller details, AST interpreter, derived ToolBench-IR evaluator metadata, compute environment, software stack, and reproduction commands needed to rerun the main comparisons.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

1130 (d) We recognize that reproducibility may be tricky in some cases, in which case
1131 authors are welcome to describe the particular way they provide for reproducibility.
1132 In the case of closed-source models, it may be that access to the model is limited in
1133 some way (e.g., to registered users), but it should be possible for other researchers
1134 to have some path to reproducing or verifying the results.

1135 5. Open access to data and code

1136 Question: Does the paper provide open access to the data and code, with sufficient instruc-
1137 tions to faithfully reproduce the main experimental results, as described in supplemental
1138 material?

1139 Answer: [No]

1140 Justification: The current anonymous submission includes only the paper PDF and does not
1141 include a public code archive or separate artifact package. The paper and supplement report
1142 the algorithm, benchmark sizes, task-subset construction rules, hyperparameters, evaluation
1143 protocol, and statistical analyses. Code, task metadata, and reproduction scripts will be
1144 released after paper acceptance.

1145 Guidelines:

- 1146 • The answer [N/A] means that paper does not include experiments requiring code.
- 1147 • Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 1148 • While we encourage the release of code and data, we understand that this might not
1149 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
1150 including code, unless this is central to the contribution (e.g., for a new open-source
1151 benchmark).
- 1152 • The instructions should contain the exact command and environment needed to run to
1153 reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 1154 • The authors should provide instructions on data access and preparation, including how
1155 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 1156 • The authors should provide scripts to reproduce all experimental results for the new
1157 proposed method and baselines. If only a subset of experiments are reproducible, they
1158 should state which ones are omitted from the script and why.
- 1159 • At submission time, to preserve anonymity, the authors should release anonymized
1160 versions (if applicable).
- 1161 • Providing as much information as possible in supplemental material (appended to the
1162 paper) is recommended, but including URLs to data and code is permitted.

1165 6. Experimental setting/details

1166 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
1167 rameters, how they were chosen, type of optimizer) necessary to understand the results?

1168 Answer: [Yes]

1169 Justification: The paper reports the task suites, train/evaluation splits where applicable,
1170 controller settings, posterior updates, and baseline configurations in the experiments section
1171 and appendix.

1172 Guidelines:

- 1173 • The answer [N/A] means that the paper does not include experiments.
- 1174 • The experimental setting should be presented in the core of the paper to a level of detail
1175 that is necessary to appreciate the results and make sense of them.
- 1176 • The full details can be provided either with the code, in appendix, or as supplemental
1177 material.

1178 7. Experiment statistical significance

1179 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1180 information about the statistical significance of the experiments?

1181 Answer: [Yes]

Justification: The results report seed-level means and standard deviations for the main comparisons and raw-backed controls. The text avoids unsupported public-leaderboard comparisons and reserves formal significance statements for the paired task-level tests in the supplement.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper now states the 4×A100-80GB setup, the 68–78 hour wall-clock range for the reported sweeps, the approximate 288–312 GPU-hour envelope, and the Python 3.10 / PyTorch 2.1 / CUDA 12.1 software stack used for reproduction.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses established benchmark tasks and does not involve deception, unsafe data collection, or non-consensual human-subject research. The submission is anonymized and is intended to conform to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- 1234
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- 1235
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).
- 1236
- 1237

10. Broader impacts

1238

1239 Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

1240

1241 Answer: [Yes]

1242 Justification: The paper discusses the potential benefits of more reliable autonomous agents and the risks of improving automated task execution, including possible misuse if deployed without monitoring or domain-specific safeguards.

1243

1244

1245 Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
 - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
 - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
 - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
 - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
 - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).
- 1246
- 1247
- 1248
- 1249
- 1250
- 1251
- 1252
- 1253
- 1254
- 1255
- 1256
- 1257
- 1258
- 1259
- 1260
- 1261
- 1262
- 1263
- 1264
- 1265
- 1266
- 1267

11. Safeguards

1268

1269 Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

1270

1271

1272 Answer: [N/A]

1273 Justification: The paper does not release a new high-risk model, scraped dataset, or general-purpose agent service. StratEx is evaluated as an algorithmic framework on existing benchmarks.

1274

1275

1276 Guidelines:

- The answer [N/A] means that the paper poses no such risks.
 - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
 - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
 - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.
- 1277
- 1278
- 1279
- 1280
- 1281
- 1282
- 1283
- 1284
- 1285
- 1286

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing benchmarks, models, and baseline methods are credited through citations in the related work and experimental sections. License and terms-of-use details for the original assets and the derived ToolBench-IR evaluation metadata will be included with the post-acceptance code and metadata release where applicable.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The proposed contribution is an algorithmic exploration framework. The paper also uses ToolBench-IR as a derived evaluation asset: a documented 200-task subset and deterministic evaluator built from existing ToolBench-style tasks for controlled mechanism analysis. The supplement documents its task IDs, construction rules, exclusion rules, evaluator assumptions, and limitations; the corresponding metadata will be released after paper acceptance.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured formats. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not conduct crowdsourcing experiments or collect new data from human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve human-subject experiments, so IRB approval or an equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLM tools were used only for writing and editing assistance. They did not affect the core methodology, scientific rigor, originality, experimental design, or reported results.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.